

DirtyRand: Data-only Attacks against the Linux Kernel RNG

Anonymous Submission — #60

Abstract

The effectiveness of many system-security mechanisms hinges on the unpredictability of a single component: the random number generator (RNG) of the OS. While decades of work hardened the cryptographic design on the Linux RNG, and ensured the quality of entropy sources, the integrity of its *internal generator state* in the presence of memory safety vulnerabilities is largely unexamined. In this work, we demonstrate that Linux’s RNG state is an unprotected, high-value target for data-only attacks. We present DirtyRand, the first data-only attack targeting the Linux RNG’s internal state. Assuming a weak, unprivileged local adversary with no kernel-memory disclosure capabilities and only a single, restricted write of attacker-known (not attacker-chosen) data, DirtyRand exploits the RNG’s deterministic, hierarchical structure; a single well-placed corruption collapses the entropy of all downstream randomness consumers across user and kernel space, rendering random draws predictable. We also introduce a follow-on primitive that halts the RNG’s periodic reseeding cycle, making the compromise persist indefinitely. We demonstrate the severity of DirtyRand through end-to-end exploits that bypass kernel and user-space stack canaries, ASLR, and SLUB hardening, and that mount DNS cache poisoning, TCP connection hijacking, and TLS premaster-secret recovery.

To mitigate DirtyRand, we design and implement RNGuard: a protection mechanism that isolates and mediates access to the RNG’s sensitive state using hardware memory-protection primitives (Intel MPK on x86-64 and ARM MTE on AArch64). Unlike the deliberately weak adversary DirtyRand assumes, RNGuard is engineered to withstand a strictly stronger attacker wielding an arbitrary (kernel-memory) read-write primitive. By applying RNGuard against our end-to-end exploits and arbitrary read scenarios, we show that RNGuard neutralizes both corruption-based attacks and direct disclosure. Furthermore, RNGuard imposes minimal overhead on macro benchmarks ($\leq 1\%$ across the Phoronix Test Suite), demonstrating that the RNG’s internal state can be fully protected without compromising system performance.

1 Introduction

Cryptographically secure *pseudorandom number generators* (RNGs) form a foundational pillar of modern OS security. The efficacy of defenses ranging from memory-corruption mitigations—e.g., Address Space Layout Randomization (ASLR) [123], stack canaries [47], and heap hardening [43, 105]—to network protocols like TLS and TCP hinge on the unpredictability of the underlying source of randomness. In the Linux kernel, a single, dedicated RNG accumulates entropy from unpredictable system events, and is used to periodically refresh a *deterministic, hierarchical* cryptographic pipeline, which services randomness requests across both user and kernel space. While a plethora of prior work studied the cryptographic strength of the Linux RNG and implications of poor entropy sources (§2.4), the *integrity of its internal state* remains unexamined.

This oversight is particularly dangerous given the current exploitation landscape. With the widespread deployment of control-flow hijacking mitigations, such as Control-flow Integrity (CFI) [8, 61, 156], adversaries have increasingly pivoted to data-only attacks [1, 4, 14, 34, 39, 41, 54, 71, 72, 78, 99, 137, 148], which corrupt non-control data to manipulate program logic while remaining within legal execution paths. Unfortunately, systematically identifying which non-control data is “security-critical” remains a notoriously difficult, open challenge. As a result, the most consequential data-only attacks have avoided the general case, identifying particular, high-impact targets one at a time: e.g., copy-on-write state in Dirty COW [1], stale pipe-buffer flags in DirtyPipe [4], and credential objects in DirtyCred [99]. We follow this line of work, but with a target that differs from the *self-contained* objects previously identified: rather than target, for example, a credential structure that elevates the privileges of a single process, we focus on the RNG’s internal state that represents a *shared root of trust*, feeding over 200 call sites across kernel subsystems. Compromising it thus invalidates the central assumption of many—otherwise unrelated—defenses at once.

We present DirtyRand: the first data-only attack target-

ing the Linux RNG’s internal state. Under a *constrained*, realistic threat model—i.e., an unprivileged, local adversary armed with only a single, restricted write of attacker-known (not attacker-chosen) data and no kernel-memory disclosure capabilities—DirtyRand exploits the deterministic, hierarchical design of the Linux RNG. By corrupting a single “sensitive” state variable, DirtyRand collapses the entropy of all downstream randomness consumers, rendering subsequent random draws predictable and the defenses that rely on them bypassable. We further introduce a follow-on primitive that maliciously manipulates the kernel’s `workqueue` subsystem to halt the RNG’s periodic reseeding, allowing DirtyRand’s state compromise to persist indefinitely.

To illustrate the severity and impact of DirtyRand, we developed a series of real-world, end-to-end exploits that leverage DirtyRand to subvert widely deployed security mechanisms and escalate privileges. Specifically, our case studies demonstrate how compromising the RNG can circumvent kernel and user-space stack canaries, ASLR, and SLUB hardening. Further, we demonstrate that DirtyRand can break network-level defenses by mounting attacks that include DNS cache poisoning, TCP hijacking, and TLS premaster-secret recovery.

Recognizing that the security of randomness-dependent mechanisms cannot hold if their root of trust is compromised, we design and implement RNGuard: a lightweight, hardware-assisted defense that isolates and mediates access to the Linux RNG’s sensitive persistent and transient states. Our implementation of RNGuard leverages Intel Memory Protection Keys (MPK) on x86-64 and ARM Memory Tagging Extensions (MTE) on AArch64 to confine sensitive-state access to legitimate RNG call paths. In contrast to the deliberately constrained adversary assumed by DirtyRand, RNGuard withstands a strictly stronger attacker armed with arbitrary kernel read–write primitives. In our evaluation of RNGuard, we demonstrate that our defense effectively neutralizes corruption-based prediction and direct state-disclosure attacks, all while imposing minimal macro-benchmark overhead ($\leq 1\%$ on the Phoronix Test Suite).

In summary, this paper makes the following contributions:

- We propose DirtyRand, the first data-only attack targeting the Linux kernel’s internal RNG state, demonstrating that a single, restricted write of known data is sufficient to compromise system-wide randomness.
- We introduce a follow-on primitive that halts the RNG’s reseeding cycle, extending the attack window indefinitely.
- We demonstrate the severity of DirtyRand through a diverse set of real-world, end-to-end exploits targeting memory-corruption mitigations and network protocols.
- We design and evaluate RNGuard, a hardware-assisted memory isolation defense utilizing MPK and MTE to comprehensively protect the RNG’s sensitive persistent and transient state from arbitrary read–write adversaries with minimal performance overhead.

2 Background and Related Work

2.1 Data-only Attacks

As the widespread deployment of mitigations like Control-flow Integrity (CFI) [8, 61, 156] makes traditional control-flow hijacking increasingly difficult, adversaries have pivoted toward data-only attacks [1, 4, 14, 34, 39, 41, 54, 71, 72, 78, 99, 137, 148]. By corrupting non-control data to manipulate program logic without wandering outside a victim program’s control-flow graph (CFG), data-only exploits circumvent standard defenses to achieve severe impacts, such as privilege escalation. Defending against data-only attacks is fundamentally challenging due to the fluid, dynamic nature of data; while various software and hardware-assisted isolation defenses have been proposed [9, 19, 25, 35, 36, 38, 48, 110, 116, 128, 129, 133, 141, 145, 158], they all share a critical limitation: they inherently assume that security-critical data can be accurately identified. This underlying problem—determining exactly *what* non-control data requires protection—remains difficult and orthogonal to the enforcement mechanisms themselves, typically resulting in incomplete manual [35, 141] or automatic [144, 172] coverage. (For a comprehensive discussion on the evolution, mechanics, and defenses regarding data-only attacks, please see Appendix A.)

2.2 (Hardware) Memory Isolation

Hardware memory isolation mechanisms were originally designed to mediate access to a shared physical address space among competing programs. Early approaches, such as segmentation [121] and paging [88], enforced this isolation at the coarse granularity of a segment or page. While finer-grained isolation has long been desirable, purely software-based enforcement remains too expensive for practical use [57, 108, 162, 169]. However, modern hardware has narrowed this gap, enabling efficient selective isolation mechanisms [40], such as Intel MPK [76] and ARM MTE [11].

Intel MPK. Intel Memory Protection Keys (MPK) [77] is a hardware mechanism providing efficient, intra-address-space memory isolation without expensive page-table modifications or permission-changing syscalls. MPK supports two variants: *Protection Keys for Userspace* (PKU), introduced on Skylake server CPUs and applicable to user-space pages, and *Protection Keys for Supervisor* (PKS), introduced more recently on server-class CPUs and applicable only to supervisor-mode pages. Both variants share the same underlying mechanism: a 4-bit *protection key* (i.e., a *pkey*) is stored in the unused bits of each page-table entry (PTE) that partitions the address space into 16 disjoint domains; the *pkey* is then used to index a 32-bit register—i.e., the per-thread `PKRU` for user space or the per-CPU `IA32_PKRS` for kernel space—which is divided into 16 2-bit slots, each encoding access/write permissions for a given *pkey* domain. MPK then extends the MMU with

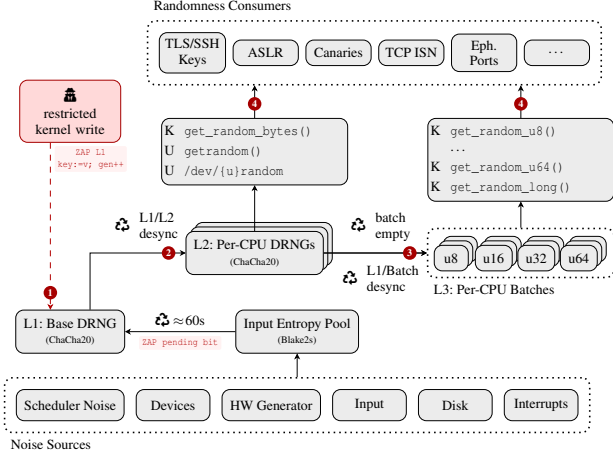


Figure 1: Overview of Linux’s RNG with the flow of DirtyRand’s corruption marked.  notes refresh-event criteria. K and U indicate kernel- and user-space API functions.

an additional check on every memory access that ensures the operation is allowed according to the permission bits. For example, if `IA32_PKRS[PTE.pkey] = 0b00`, reads and writes are permitted; `0b11` disables both.

ARM MTE. ARM Memory Tagging Extensions (MTE) [11] is a hardware memory-safety feature introduced in ARMv8.5-A. MTE partitions memory into 16-byte granules that are each associated with a 4-bit *allocation tag*, which is maintained by hardware and stored in a separate memory region reserved for allocation tags. Pointers also carry a 4-bit *address tag* in their top byte via ARM’s Top Byte Ignore (TBI) feature. On every load and store, the CPU compares the pointer’s address tag against the target memory granule’s allocation tag; on a match, execution continues normally, else a fault is raised. Notably, faults can be delivered synchronously or asynchronously, with the former being preferable for security-critical settings [30].

Originally, MTE was designed to detect memory-safety violations, such as use-after-free bugs and out-of-bounds accesses. Under MTE, a “stale” or out-of-bounds pointer will collide with a memory granule’s tag with a probability of 1/16, making detection probabilistic but cheap. Recently, however, MTE has also been repurposed as a general isolation mechanism; a thread holding exclusive tag ownership can segregate its memory, blocking other threads’ access [110, 116].

2.3 Linux Kernel Random Number Generator

Modern OSes rely on cryptographically secure random number generators (RNGs) to support a broad range of security-critical functionality, spanning cryptographic key generation [46, 163]; memory-safety-related mitigations, e.g., Address-Space Layout Randomization (ASLR) [24, 122, 168], stack canaries [47], ARM Pointer Authentication Code (PAC) keys [98, 130], and SLUB freelist randomization [62]; eBPF JIT constant blinding [29]; network protocols,

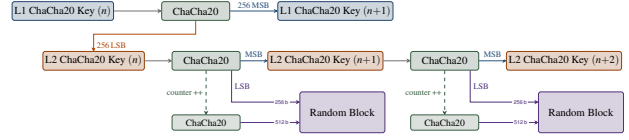


Figure 2: ChaCha20 overview.

e.g., DNS source-port randomization [73] and TCP initial-sequence-number generation [64]; and authentication protocols [140, 167]. A secure RNG must produce outputs that are computationally indistinguishable from uniform random bits, and must further provide forward and backward secrecy [18]: compromise of the generator state should reveal neither previously emitted outputs (forward secrecy) nor outputs produced after sufficient fresh entropy has subsequently been mixed into the state (backward secrecy).

In the Linux kernel, randomness is provided by a dedicated RNG subsystem that serves both user space and the kernel itself; Figure 1 illustrates the high-level structure of the Linux RNG. The subsystem consists of an input entropy pool and a three-tier, randomness-generation pipeline composed of a base deterministic random number generator (DRNG), a set of per-CPU DRNG instances, and a set of per-CPU output batches, which we refer to as the L1, L2, and L3, respectively. The pipeline mirrors a cache hierarchy (in reverse): the L1 serves as the root source of randomness, while the L2 and L3 satisfy per-CPU requests so as to avoid contention on the global L1 source. Entropy harvested from system activity is accumulated in the input pool and periodically used to (re)seed the DRNG hierarchy; user- and kernel-space consumers then draw randomness from the hierarchy through standard interfaces, e.g., the `getrandom()` system call and the `get_random_bytes()` in-kernel API. The following subsections describe each component in greater detail.

2.3.1 ChaCha20

The cryptographic primitive the Linux kernel uses to generate *pseudorandom output* across L1–L2 is the ChaCha20 stream cipher [23, 119]. ChaCha20 is a 20-round Add–Rotate–XOR (ARX) construction parameterized by a 256-bit (32-byte) key, a 96-bit (12-byte) nonce, and a 32-bit (4-byte) block counter; each invocation of ChaCha20’s block function emits 512-bits (64-bytes) of output stream. Both the L1 and L2 maintain a persistent ChaCha20 state consisting solely of a 32-byte key. On each randomness draw from the L1 or L2, the kernel constructs a *transient* ChaCha20 state on the stack using just the stored key (both the nonce and the block counter are set to zero), invokes the block function, and—when more than one block of output is required—increments the counter across successful invocations. Once a random draw is complete, the kernel discards the entire transient state. The L3 batches are filled similarly from L2 outputs.

Figure 2 illustrates how the kernel chains successive

ChaCha20 invocations across the hierarchy, following Bernstein’s *fast-key-erasure* construction [22]. On every invocation, the upper 256 bits of the 512-bit key stream are taken from the output and written back to the issuing DRNG instance as its new key, clobbering the old key; the lower 256 bits are made available to the output stream. For the L1 base DRNG, the lower bits are used as the key for a given L2 instance, propagating fresh randomness one tier down. For the L2 per-CPU DRNGs, the lower bits form the initial randomness output that can be extracted by a consumer. If more than 256 bits of randomness are required, the output of additional ChaCha20 invocations—differing in only their incremented counter—are concatenated together to fill the remaining output. Crucially, overwriting the ChaCha20 key at every step is how the Linux RNG attains forward-secrecy: a state compromise at time t yields no information about outputs produced before t , since the keys that produced output prior to t were used in isolation and no longer exist.

2.3.2 Input Entropy Pool

The kernel RNG relies on *entropy*—unpredictability derived from non-deterministic events—to produce random values. Entropy is collected from a variety of *noise sources* (see Figure 1 for examples), which are measurable phenomena that are partially or fully unpredictable to an observer. Entropy contributions from a configurable set of such sources are mixed into an *input entropy pool* that serves as a consolidated reservoir of unpredictability for the rest of the RNG subsystem.

The input entropy pool is implemented as an incrementally updated BLAKE2s [13] hash over the existing state of the input pool, mixed with fresh entropy contributed by noise sources [117]. Notably, the purpose of the input pool is not to directly generate random outputs, but to accumulate entropy in a way that is unpredictable to an observer, such that it can then seed the downstream L1 DRNG. In Linux v6.12.11, at steady state, this reseed cycle fires every 60s, ensuring fresh randomness is continuously pushed downstream for backward secrecy [18]. Only the L1 DRNG is reseeded by the input entropy pool: L2 is seeded from L1 and L3 is filled by L2.

2.3.3 The L1–L3 Hierarchy

L1 Base DRNG. The entropy accumulated in the input pool is consumed by the L1 base DRNG, which sits at the top of the kernel RNG hierarchy. Its persistent state is small—a 256-bit ChaCha20 key together with a `generation` counter that lets L2 and L3 instances determine when they have fallen out of sync with L1. (See Appendix C for L1’s declaration.) The L1 is invoked on demand internally to re-key the L2 per-CPU DRNGs using the low 256 bits of ChaCha20 output—the high 256 bits are used to perform fast-key-erasure [22], overwriting the current L1 key to provide forward secrecy [18]. The L1 is also reseeded with fresh entropy every 60s via the input

entropy pool and its `generation` incremented.

Notably, the ChaCha20 nonce and block counter are not part of L1’s persistent state—they are zeroed prior to every new ChaCha20 block operation and exist only on the calling thread’s stack (§2.3.1). Because ChaCha20 is deterministic given its inputs, the entire downstream output of the kernel RNG is fully determined by the current value of `key`: an attacker able to leak or control `key` can predict every random byte produced by the kernel RNG until the next reseed cycle.

L2 Per-CPU DRNGs. The kernel maintains one L2 DRNG instance per online CPU. Structurally, the L2 DRNG is identical to the L1 DRNG (i.e., a 256-bit ChaCha20 `key` and a `generation` counter) but it is locked with a per-CPU `local_lock_t` rather than a global spinlock. The L2 DRNG instances are directly—or indirectly by filling the L3 batches—responsible for filling all randomness requests; e.g., invocations of `get_random_bytes()` from in-kernel callers, reads from `/dev/{u}random`, and `getrandom()` system calls. Importantly, a given L2 DRNG is re-keyed from the L1 base DRNG when their `generation` counters mismatch. As a result of a given L2’s `key` being directly descended from the L1 `key`, a compromise of the L1 base DRNG state propagates to all downstream L2 DRNGs on the next re-key. This lazy, mismatch-driven re-key amortizes the L1’s 60-second reseed cycle across all CPUs: the only cross-CPU contention that *may* result from the kernel RNG is during the brief window, after an L1 reseed, when the first randomness request from each CPU causes an L2 re-key.

L3 Per-CPU Batches. The kernel also maintains *per-CPU batched entropy pools* (L3) where small buffers of randomness are filled via the L2 DRNGs and drained to service in-kernel requests for fixed-size randomness. As a result, a request for a single integer is serviced quickly, avoiding invoking ChaCha20. Separate batches are maintained per integer width for each online CPU: i.e., `get_random_{u8, u16, u32, u64}()`. (See Appendix C for L3’s declaration.) The entropy buffer is sized per integer type so that it is always 96 bytes, or 1.5 ChaCha20 blocks—making a full refill of a batch’s entropy buffer two ChaCha20 block invocations with the local L2 (the first invocation performs fast-key-erasure and only emits 32 bytes to the output stream). Similar to the L2 state, the L3 maintains a per-CPU local lock and a `generation` counter that determines when a given L3 batch is out of sync with the L1 base DRNG. Finally, the L3 state also has a `position` index indicating where in the entropy buffer to pull the next random value from. After reading a value at `entropy[position]`, the memory is zeroed and `position` is incremented. Once `position` reaches the end of the entropy buffer, the latter gets refilled from L2 and `position` is zeroed.

2.4 Attacks against the RNG

The Linux RNG has previously been the target of three main classes of attacks: entropy starvation, entropy degradation,

and state compromise. Entropy starvation influences *entropy collection*: preventing sufficient entropy collection at boot-time yields weak TLS and SSH keys [70], predictable canaries and IPv6 IDs [84], and vulnerabilities across mobile [51, 89], embedded [97], and virtualized [58, 138, 161] systems. Entropy degradation directly targets the quality of collected entropy to reduce unpredictability. For example, frequency injection [106], temperature control [146], entropy pool corruption [17], and predictable entropy contributions [53, 94] can all render an RNG’s output guessable. Finally, state compromise leverages the exposure of a given RNG state to infer past or future outputs [18, 42, 52, 69, 96]. (For an extended description of RNG attacks, please see Appendix B.) Crucially, in all these cases, the adversary starves the generator, manipulates its noise sources, or performs cryptanalysis on its output; *none treat the generator’s internal cryptographic state as a target of corruption*.

3 Threat Model

Adversarial Capabilities (Attack). We consider an unprivileged, local attacker who aims to corrupt the state of the Linux kernel’s RNG as a central step of a multi-stage exploit—e.g., to escalate privileges [27, 143] or to mount network-based attacks, such as session hijacking and spoofing [20, 91]. The attacker wields a one-shot, restricted write primitive targeting up to 33 bytes—e.g., a 32-byte ChaCha20 key plus 1-byte of the proceeding generation field—of contiguous memory in the kernel’s `.data` section. The attacker chooses *where* to write but not *what*: the written values are fixed and outside the attacker’s control, though the values are known to the attacker in advance (e.g., a write of all zero bytes). *Crucially, the attacker has no kernel read capability*. This is a deliberate scoping choice, not a limitation of our threat model. In our setting, a memory-disclosure primitive is strictly more powerful than a restricted write, because: (1) directly reading the RNG state already violates the confidentiality it is required and assumed to provide [52, 55, 69], and (2) bypassing defenses that rely on secret data with only a write primitive is significantly harder, noisier, and more complex than doing so with a read [27, 147]. By deliberately confining the attacker to the weaker primitive, our attack establishes a strict lower bound on what an adversary can achieve. Memory-corruption-based information leaks are therefore out of scope; however, we assume the addresses of `.data` structures are already known via a prior KASLR bypass that does not rely on memory-corruption-based reads [31, 32, 59, 68, 74, 79, 80, 82, 90, 93, 100–103, 139, 164]. Finally, we do not restrict the kernel’s entropy sources: hardware generators such as RDRAND/RDSEED [75], interrupt timings, and other noise inputs are all assumed to function correctly and seed the RNG as intended. Our attacker neither focuses on weakening nor predicting these inputs, instead corrupting *post-reseed* state—a more conservative threat model than prior work that targets entropy sources directly [132].

Adversarial Capabilities (Defense). The defense against DirtyRand assumes a much more capable attacker to demonstrate its robustness. For this setting, we assume an unprivileged, local attacker armed with arbitrary read and write primitives in kernel memory that can be used whenever and however many times the attacker needs; there are no restrictions on the size of data read or written. Under this setting, an attacker can carry out the attack described in this work, or short-cut the process by reading the secret RNG state directly. **Hardening Assumptions.** We assume a modern Linux kernel ($\geq v6$) with the standard hardening schemes enabled. This includes, but is not limited to: support for non-executable memory [104, 112] correctly enforcing a W^X memory protection policy [120] (i.e., code injection [85] is not possible), stack smashing protection, and control-flow integrity [56, 61]. As noted above, we assume that KASLR has already been bypassed—a realistic assumption for our adversary given the myriad works that break KASLR, even without a (memory-corruption-based) read primitive.

Overall, our threat model is on par with state-of-the-art data-only attacks and attacks against RNGs [39, 42, 58, 70, 72, 78].

4 Attack Strategy

DirtyRand exploits two properties of the kernel RNG: #1 the determinism of its components, and #2 its hierarchical structure (see ① – ④ in Figure 1). A ChaCha20-based construction expands the L1 DRNG’s single, 32-byte key into keys for the L2 per-CPU DRNGs, which in turn fill the L3 per-CPU batch pools (§2.3). The L2 DRNGs and L3 batch pools then feed user- and kernel-space API functions/syscalls. The L1 DRNG is thus a single point of failure: one well-placed write of an attacker-known value—which we call a *zap*—collapses the entropy of *every* downstream consumer. A key overwrite alone, however, does not take effect immediately: the state of the L2 and L3 are lazily updated only when the L1 generation counter advances. DirtyRand addresses this by also incrementing the counter (positioned directly after the overwritten key field), after which all output—e.g., `get_random_bytes()`, `getrand()`, and the `get_random_u{8, 16, 32, 64}()` batch interfaces—becomes a deterministic function of the known state. Notably, the same primitive applies further down the hierarchy as well: the L2 DRNGs and L3 batch pools can be zapped directly, widening the attack surface and providing several interchangeable injection points.

The remainder of this section details the core technique of zapping RNG state and tracing its propagation through the hierarchy (§4.1), followed by showing how to lengthen the attack window by halting the reseed cycle indefinitely (§4.2).

4.1 Corrupting and Predicting RNG State

Emulating the RNG. In order to predict the “randomness” emitted at *any* level of the kernel RNG hierarchy, we cre-

ated a user-space library/analogue that exactly reproduces the kernel’s ChaCha20 construction. Seeded with a known state (i.e., the ChaCha20 key)—such as the attacker-known value used for zapping—the library replays the exact byte stream the kernel will subsequently produce.

To predict n successive invocations of the L1 DRNG from a given state, the library performs n rounds of ChaCha20 with fast key erasure [22]: each iteration uses the current L1 key to generate one ChaCha20 block, the first half of which overwrites the L1 key (for fast key erasure) and the second half of which is that invocation’s output. Notably, for requests larger than 32 bytes, the kernel continues with the same ChaCha20 stream (with an incrementing block counter rather than re-keying); our library mirrors this so both small and large requests are reproduced exactly. To predict a given L2 DRNG’s output from a starting L1 state, the library first performs one L1 ChaCha20 operation to derive the L2 key and then performs ChaCha20 with fast key erasure using L2’s key to emit the L2 output stream. Finally, because the L3 batch pools are simply cached L2 output—replaying the corresponding L2 operations also recovers every L3 value.

Corrupting the L1 DRNG. Every 60 seconds the L1 DRNG is reseeded: it extracts fresh entropy from the input entropy pool, stores it as its new ChaCha20 key, and increments its generation counter. This new key is later expanded via ChaCha20 with fast key erasure to re-key the downstream L2 per-CPU DRNGs (lazily, the next time a given L2 DRNG is used). Therefore, corrupting the L1 DRNG lets an attacker influence *every* downstream consumer (see Figure 1). Targeting L1 is straightforward: its state is global, residing in the kernel’s writable `.data` section at a constant offset from the kernel image base, making its address discoverable offline without any runtime leak (assuming KASLR is bypassed; §3).

In an DirtyRand attack against the L1 DRNG, an adversary armed with a restricted write corrupts the 32-byte key field of L1’s ChaCha20 state. Importantly, the adversary does not need to control the bytes written, they only need to know their value. After the overwrite, L1’s output, and that of every downstream consumer, can be replayed offline with our library/analogue. However, an L1 key overwrite alone is inert until a given L2 DRNG detects an L1–L2 generation mismatch and attempts a re-key from the L1 DRNG. Typically, the generation counters desynchronize after a reseed cycle (every 60 seconds) since this increments L1’s generation counter. Thus, to force the propagation of known state, the attacker can either: (1) time the overwrite to occur just after a reseed but before the L2 DRNGs (lazily) observe the desync, or (2) desync them directly. Option (2) is preferable as it decreases exploit complexity and increases exploit reliability: since generation sits after the key field in memory, a single 33-byte write (i.e., 32 bytes to overwrite key and 1 byte for generation) corrupts the key and guarantees propagation.

The length of the attack window depends on write timing. With the input entropy pool $\rightarrow$ L1 reseed cycle occurring

every 60s, an adversary has from the moment of the write until the next reseed to act. In many use cases this is sufficient time to exploit the inferred “random” output. However, we show in Section 4.2 that reseeding can be halted indefinitely, persisting the corrupted state and giving greater attack flexibility.

Targeting the L2 DRNGs. The L2 and L3 levels are not merely conduits for a corrupted L1 state—each is an independent corruption target with its own trade-offs. An L2 DRNG is essentially a CPU-local replica of the L1 base DRNG except its output: (1) services randomness requests from `get_random_bytes()`, `getrandom()`, and reads of `/dev/{u}random`, and (2) fills the L3 batch pools (§2.3.3). Overwriting an L2 key therefore makes that CPU’s stream—and the batches and API calls it feeds—predictable. However, because the L2 state is per-CPU, the attacker must corrupt (or pin) the exact CPU that will service the randomness request. In contrast, corrupting the *global* L1 base DRNG sidesteps this issue entirely, making it the preferred target.

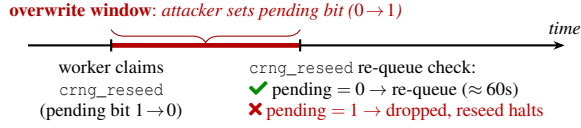
Targeting the L3 Batch Pools. The L3 batch pools expose the cheapest, most surgical primitive. A batch is a non-cryptographic, per-CPU, per-width (i.e., u8, u16, u32, u64) cache of L2 DRNG output, refilled when its position reaches the end of the buffer or its generation mismatches L1’s. Beyond inheriting predictability from a corrupted L1 or L2, a batch pool can be exploited in two main ways: (1) an attacker can directly overwrite the cached randomness with known values, or (2) an attacker can perform a single-byte overwrite of position to rewind it to an already-consumed slot using no key knowledge and no offline replay—because the kernel zeroes slots after use, the next randomness request will return 0. However, both (1) and (2) are temporary, requiring subsequent overwrites with each batch pool update. As a result, we consider corrupting the L1 base DRNG to be the strongest and most flexible primitive.

4.2 Halting the Reseed Cycle

Section 4.1 demonstrates that an attacker can corrupt the state of the L1 base DRNG to predict the kernel RNG’s outputs. This control, however, is short-lived: the kernel’s L1 base DRNG is reseeded every 60s from the input entropy pool, replacing its internal state—`L1.key`—with fresh entropy to restore unpredictability. Thus, a single corruption grants predictability only until the next reseed—a window of *at most* 60s. To increase attacker control and flexibility, we present a follow-on attack that halts the reseed cycle entirely.

The kernel relies on the workqueue subsystem to defer and asynchronously execute background tasks. Deferred work is wrapped in a `delayed_work` structure, which starts a timer that enqueues the contained `work_struct` on expiry; self-renewing tasks, such as `crng_reseed`, restart this timer from within their callback. To prevent double-enqueueing, the kernel sets a pending bit in the `work_struct`’s data field on enqueue and clears it only when a worker claims the item for

execution. Re-queueing tests this bit: if it is already 1, the kernel treats the request as a duplicate and *silently drops it*. An attacker who sets the bit while `crng_reseed` is executing therefore suppresses its self-renewal:



With the re-queue dropped, L1 is never refreshed again, letting the attacker’s initial state compromise dictate all future outputs. The bit’s location is fixed: `crng_reseed` declares its `delayed_work` as a function-scope `static`, so it resides in `.data` at a link-time-constant address and requires no additional disclosure to locate. Its *value*, however, packs adjacent flags that must remain semantically valid (Appendix D).

Timing Side Channel. Winning this race, however, requires precise timing: the overwrite window spans only a few microseconds. We therefore develop a timing side channel that identifies when a reseed occurs and predicts subsequent ones, letting the attacker concentrate corruption attempts in a window *orders of magnitude smaller* than the 60s reseed interval.

The side channel exploits the RNG’s determinism. After zapping `L1.key`, we pre-compute the sequence of `getrandom()` outputs the kernel will produce (§4.1), then poll `getrandom()` from user space and match each returned value against it. The outputs track the prediction until a reseed introduces fresh entropy, at which point the hierarchy updates and the sequence diverges, giving an unambiguous signal. Once the system reaches steady state, reseeds recur at highly regular intervals with minimal jitter, so detecting one reseed predicts the rest accurately, narrowing the attack window from 60s to a few milliseconds.

5 Case Studies

System Setup. We carried out the case-study exploits in QEMU/KVM virtual machines (VMs) running atop an Intel Xeon W-2145 CPU. Each VM was provisioned with 16GB of memory, 4 CPUs, networking support (with the default `e1000` NIC), and a Debian v12 (“Bookworm”) root file system. All VMs ran Linux kernel v6.12.11 with a default configuration; KASLR was bypassed for all case studies (§3). For the networking case studies that involve multiple, connected VMs (e.g., a client, server, and off-path attacker), we model a realistic topology where each VM resides on a different subnet and inter-VM traffic is routed through the host via IP forwarding.

Restricted-write Primitive. We developed a real-world, kernel-memory write primitive based on CVE-2022-34918 [134], which we forward-ported to Linux kernel v6.12.11 [15]. The primitive leverages a heap-buffer overflow in the kernel’s Netfilter (`nf_tables`) subsystem to corrupt the `struct page` pointer of an adjacent `struct pipe_buffer`, turning a plain pipe write from user space into an

Table 1: Summary of end-to-end case studies. *Level* indicates which level of the RNG hierarchy supplies the predicted value.

Case study	Predicted value	Level	Outcome
Canaries (user)	AT_RANDOM	L2	mitigation bypass
Canaries (kernel)	task canary	L3	mitigation bypass
ASLR	stack base	L3	mitigation bypass
SLUB freelist	s->random	L3	mitigation bypass
DNS	ephemeral port, TXID	L2, L3	cache poisoning
TCP	ephemeral port, ISN	L2, L3	RST hijack
TLS	GnuTLS PRNG seed	L2	PMS recovery

attacker-controlled, physical-memory write [81]. By aiming the `struct page` pointer at the physical memory backing the kernel’s `.data` section, we can corrupt the RNG’s persistent state. While this primitive grants flexible (kernel) memory corruption, we only use it to *simulate a restricted write inline with our threat model* (§3), without loss of generality.

Validating Prediction. As a baseline test, we zapped `L1.key` and predicted the output of `getrandom()` (user-space API, draws from L2), `get_random_bytes()` (kernel API, draws from L2), and `get_random_long()` (kernel API, draws from L3), invoking the latter two from a test module since they are not callable from user space. In every case, the extracted values matched our predictions exactly. Table 1 summarizes the six end-to-end case studies that follow.

5.1 Memory-corruption Mitigations

5.1.1 Stack Canaries

Stack canaries are used to mitigate stack-smashing attacks by placing a random value—unknown to an attacker—before return addresses on the stack [47]. Before functions return, the canary value is checked for any change that would indicate memory corruption. Thus, when an adversary tries to overwrite a return address via, say, a buffer overflow, they need to know the value of the canary to incorporate into their payload, else their attack will be detected. Canaries are used in both user space and kernel space, and since the canary value is random, extracted from the kernel’s RNG, it is susceptible to DirtyRand. Notably, even more fine-grain variants of stack-canary mitigations such as DynaGuard [125] and PESC [150] are also vulnerable to DirtyRand.

Randomization Mechanism. In user space, `execve()` places `AT_RANDOM`—a 16-byte value drawn from an L2 per-CPU DRNG via `get_random_bytes()`—in the auxiliary vector, and `ld.so` copies 8 bytes of it into the TLS region at `%fs:0x28`. In the kernel (`CONFIG_STACKPROTECTOR`), `get_random_canary()` draws 64 bits from the L3 batch pools once at task creation and stores them in the `task_struct`. On context switch, the incoming task’s canary is written to a fixed, per-CPU location (e.g., `%gs:<offset>` on x86-64) for compiler-generated prologues to use.

Exploit Technique. We demonstrate that corrupting `L1.key` before a user or kernel canary is drawn renders it predictable.

- *User space:* We exploit CVE-2015-1817 [2], a stack-

based buffer overflow in `musl libc`'s [6] `inet_pton()`, which translates IPv6 addresses from text to binary form. We built `musl v1.1.7` with support for stack canaries and triggered the vulnerability through `ping` [7] (`iputils v20240905`)—a `setuid root` binary that passes its positional argument directly to `inet_pton()`. Zapping `L1.key` before `execve()` predictably fixes `AT_RANDOM`; thus, we compute the canary and `musl`'s base address (§5.1.2) offline and craft a payload that passes the stack-protector check and ROPs a `root` shell.

- **Kernel space:** We exploit a kernel module with a *synthetic* stack-based buffer overflow that copies attacker input into a fixed-size buffer without bounds checking. Zapping `L1.key` makes the L3 batch pools—and thus the task canary—predictable, even when it is re-randomized every syscall [150]. Our payload writes the expected canary followed by a ROP chain that invokes a credential-update routine to achieve `root`.

5.1.2 ASLR

Address space layout randomization (ASLR) [123] is a widely adopted hardening mechanism that introduces randomness into the virtual address space of user-space processes. In modern Linux distributions, ASLR is enabled by default [153, 157]. It randomizes the locations of major memory regions, including the stack, the `mmap` region (including dynamically loaded libraries), the `vDSO` region, the main executable, and the `brk` memory area. By adding a level of uncertainty to addresses, ASLR mitigates attacks that assume a given address-space layout. However, ASLR randomizes only the *base address* of given program's segments, not their internal layout—discovering a single address is typically sufficient to recover the base address of its memory region, bypassing ASLR.

Randomization Mechanism. When a user-space program is executed, the kernel shuffles the layout of the process' address space, drawing randomness from the kernel RNG. The equations for randomizing different address space regions, in addition to stack jitter, are shown in Table 5 in Appendix F. Except for stack jitter, random numbers are drawn using `get_random_long()`, which pulls 64 random bits from the L3 batch pools; the random draws are then permuted to make them applicable to the address-space layout.

Exploit Technique. We build a local privilege-escalation exploit based on “Looney Tunables” [131] (i.e., CVE-2023-4911 [5]): a vulnerability discovered by Qualys in the GNU C Library's dynamic loader. Specifically, a malformed `GLIBC_TUNABLES` environment variable [60] triggers an out-of-bounds write in `ld.so`'s BSS that replaces `l_info[DT_RPATH]`—the loader's pointer to metadata naming the library search path—with a stack address pointing at a forged entry. As a result, a `setuid` binary can be made to load an attacker-controlled shared object that drops a `root` shell. Step by step:

- **Zap and compute:** The attacker corrupts `L1.key` before launching the target `setuid` binary, making L3 output—and thus the per-process stack base the kernel derives during

`execve()`—a value computable offline.

- **Forge library search path:** The exploit then writes the now-known stack address into `l_info[DT_RPATH]`, pointing to a forged library search path. By placing an attacker-controlled, spoofed, `DT_NEEDED` library in the forged search path that executes a constructor, the attacker drops a `root` shell.

In Qualys' write-up, the address of the forged structure is located by a noisy, brute-force loop that reportedly takes between 30 seconds and 5 minutes to complete: each iteration writes the same fixed stack address (i.e., the midpoint of the randomized stack region) into `l_info[DT_RPATH]` and re-executes the target, repeating until ASLR happens to place the forged structure at the guessed location. Under `DirtyRand`, we turn this into a deterministic, one-shot exploit.

5.1.3 SLUB Freelist Hardening

The SLUB allocator tracks a slab's free slots with a *freelist*: a singly linked list whose `next` pointers are stored *inside* the free objects themselves. Attackers routinely overwrite these pointers—via out-of-bounds writes [66, 118] and use-after-frees [124, 170]—so that a later allocation returns an attacker-chosen address. To mitigate this, the kernel employs *freelist pointer encoding* [43, 105] to obfuscate each freelist pointer by mixing it with a per-cache random secret, degrading reliable freelist corruption primitives into blind guesses.

Randomization Mechanism. When freelist hardening is enabled (`CONFIG_SLAB_FREELIST_HARDENED`), the SLUB allocator stores each `next` pointer as `ptr ⊕ s->random ⊕ swab(ptr_addr)`. An attacker who can locate the `next` pointer's address can compute `swab(ptr_addr)`, leaving the per-cache secret, `s->random` as the only unknown. `s->random` is drawn once per cache in `do_kmem_cache_create()` via `get_random_long()` to obtain 64 random bits from the L3 batch pools. Notably, caches owned by loadable modules draw their secret at module-load time—a moment an attacker can induce to force the draw to occur after the RNG is already compromised. Thus, forging a freelist entry reduces to recovering a single 64-bit RNG output.

Exploit Technique. We build a cross-cache privilege-escalation exploit around CVE-2017-8824 [3], a use-after-free in the Linux kernel's DCCP implementation that yields disclosure of a dangling pointer and a constrained, 8-byte read-write primitive. Notably, recovering an object address and reclaiming the freed slot to obtain a restricted write are standard products of kernel use-after-free exploitation [171], and prior work derives them semi-automatically [166]; we assume no arbitrary read or write. While the intricate mechanics of cross-cache page reclamation required to bridge the vulnerable object cache with `cred_jar` are detailed in Appendix E, the core escalation proceeds as follows:

- **Alias a credential:** The attacker grooms the heap and forces `cred_jar` to reclaim pages returned to the buddy allocator, aliasing their dangling pointer with a live `struct cred`.

- *Forge the freelist*: Because the vulnerable cache was initialized after the L1 DRNG was zapped, DirtyRand is able to derive `s->random`. Using the constrained writes, the attacker overwrites the freelist head with an encoded link to the target credential and terminates the list. Both pointers decode legally without triggering mitigations.

- *Escalate privileges*: A subsequent DCCP connection requests a fresh object, which the allocator services from the forged freelist, returning the target credential object. During initialization, the kernel itself inadvertently sets `cred->uid` to 0. The child process owning that credential observes `getuid() == 0` and executes a `root` shell.

5.2 Network Protocols

5.2.1 DNS-related Randomization

Because DNS over UDP is connectionless and unauthenticated, resolvers cache the first plausible reply to an outstanding query to avoid repeated lookups [113, 114]. An *off-path* attacker can thus poison this cache by racing to inject a forged response before the legitimate one arrives [21, 83]. To succeed, the forged reply must match the query’s question section—which is largely predictable—and correctly guess two 16-bit random values: the query’s transaction ID (TXID) and the ephemeral UDP source port [73]. Thus, the security of DNS caching relies entirely on these two RNG-derived secrets.

Randomization Mechanism. A DNS query’s TXID and source port are derived from different kernel RNG APIs. We target `systemd-resolved` [152], the default DNS resolver and cache on modern Ubuntu releases [33, 151]. It draws each 16-bit TXID in user space via `getrandom()`—and thus from an L2 per-CPU DRNG. The source port is allocated in the kernel by `udp_lib_get_port()` from a single `get_random_u32()` call that draws 32 bits from the L3 batch pools, which is then deterministically reduced to a 16-bit port.

Exploit Technique. Beyond the unprivileged foothold assumed throughout (§3), this attack requires only that the adversary can deliver UDP packets to the victim from an off-path position, without observing the outbound query. The victim runs `systemd-resolved` and is about to resolve a target name, e.g., `github.com`. Having zapped `L1.key`, every subsequent random draw on the victim is a deterministic function of the planted key, and the attack proceeds as follows:

- *Trigger and predict the query*: The victim issues a DNS query, causing `systemd-resolved` to draw a TXID and the kernel to allocate an ephemeral source port. Since `L1.key` is known, the attacker computes both offline.

- *Win the race*: The attacker sprays crafted UDP responses at the predicted source port, each carrying the predicted TXID and a spoofed IP address of the legitimate upstream resolver. The first response to arrive wins, and the spray reliably beats the single legitimate reply.

The poisoned record then sits in `systemd-resolved`’s

cache for its TTL, so every subsequent lookup of the target name on the victim host resolves to an attacker-chosen IP—e.g., `github.com` resolves to a server under attacker control.

5.2.2 TCP-related Randomness

In a TCP hijacking attack, an off-path attacker injects forged TCP segments into an already-established TCP connection [20]. The two most common variants are *RST injection* and *data injection*. In RST injection, the attacker forges a segment with the RST (reset) flag set; if the receiver’s kernel accepts it, the connection is torn down and the application is left with a broken socket. In data injection, the attacker forges a segment carrying an attacker-chosen payload; if accepted, the payload is delivered to the receiving application as if it had come from the legitimate peer. For either variant to succeed and a forged TCP segment to be successfully received, two checks must pass on the victim machine: (a) the source/destination IP and port pairs must match the open connection’s 4-tuple, and (b) the sequence number of the forged segment must fall inside the receiver’s currently advertised window. For (a), if the attacker is targeting a client-server connection, where the server is running a well-known service (e.g., HTTP on port 80, HTTPS on port 443), finding out the source (i.e., the server) IP and port and the destination IP is trivial. Predicting the ephemeral destination port and the initial sequence number (ISN) is more challenging as the client’s kernel randomizes these per-connection.

Randomization Mechanism. The kernel derives the ephemeral source port from a per-call SipHash [12] over the source IP, destination IP, destination port, and the current jiffies (10s resolution), keyed by `net_secret`—a 128-bit key sampled once via `get_random_bytes()` on first use and never refreshed. The hash’s low bits index a 256KB array, `table_perturb`, which is itself initialized once via `get_random_bytes()`; the indexed slot is added to the hash’s high bits and reduced modulo the ephemeral port range to give the starting offset, after which a linear probe selects the port. Committing a port bumps that slot by the probe count plus a small `get_random_u32_below()` jitter, so successive connections within the same jiffies bucket start probing at different offsets. A TCP connection’s ISN is made by adding the hash of the 4-tuple using the same SipHash construction—keyed by the same `net_secret`—to a 64ns-resolution counter.

Exploit Technique. Both globals—`net_secret` and `table_perturb`—are lazily initialized on first use and reside in the kernel’s BSS. A `L1.key` zap *before* the system’s first connection makes both deterministic functions of the planted key; however, in the more common case, the globals were already initialized from a clean RNG, so we use the same write primitive to instead target them directly: 16 bytes for `net_secret` and the 4 bytes for the single `table_perturb` slot indexed for the target 4-tuple. While we tested both approaches, we only describe the latter since it is more agile and markedly

different from the other case studies. The victim opens a “keep-alive” TCP connection to an Nginx server, issues one HTTP request, sleeps for 30s, and issues a second request. A successful exploit injects a TCP segment with the RST flag set during the sleep window, tearing down the connection so that the second request fails. Specifically:

- *Predict and zap*: On the victim, the attacker computes the kernel’s SipHash port and ISN values using attacker-known values for `net_secret` and `table_perturb[index]`, and then invokes the restricted-write to set the globals to the known value. The next `connect()` will consume the values.
- *Open connection*: The client opens a keep-alive connection to the server, the kernel draws the predicted ephemeral source port and ISN, and the client sleeps, opening a spray window.
- *Spray RST segments*: From off-path, the attacker sends 100K RST segments to the predicted source port, spoofing the server’s IP and walking sequence numbers across the receive window—notably, the spray absorbs the residual variability from the 64ns ISN counter. When one segment lands in the correct window, the kernel accepts the segment, obeys the RST flag, and tears the connection down.

5.2.3 TLS

Transport Layer Security (TLS) ensures the confidentiality, integrity, and authentication of application data [44, 109, 136] by establishing a shared, 48-byte *premaster secret* (PMS) during the initial handshake. Because all subsequent session keys are derived from this single fresh secret, an adversary who recovers the PMS can fully decrypt and forge the session’s traffic. While key exchange mechanisms like ephemeral Diffie–Hellman require compromising the randomness of both endpoints to recover the PMS, static RSA key exchange in TLS 1.2 [136] relies entirely on the client to generate the secret. Thus, a client with predictable randomness surrenders its PMS—and the security of the entire TLS session—without requiring server compromise. While TLS 1.2 was superseded by TLS 1.3 [135] in 2018—requiring the use of key exchange mechanisms providing forward secrecy—static RSA remains widely accepted: of the top 10K sites in Tranco’s list [127], 4,072 still complete static-RSA handshakes.

Randomization Mechanism. We consider GnuTLS [107] with its default cryptographic backend, Nettle [115]. For a static RSA key exchange, GnuTLS fills a 48-byte PMS with pseudorandom bytes and overwrites the first two bytes with the advertised protocol version, leaving 46 random bytes. These random bytes come from a per-thread, ChaCha20-based PRNG built atop Nettle’s primitives. On first use, GnuTLS keys the PRNG with 32 bytes from `getrandom()` and sets an 8-byte nonce from a second-resolution timestamp, reseeding only after a `fork()`, 2 hours, or 2MB of output. The PMS is thus a deterministic function of one kernel-supplied key, a timestamp, and a ChaCha block counter—if the kernel RNG is compromised prior to seeding GnuTLS’ PRNG, every PMS

the process produces becomes predictable.

Exploit Technique. Our topology consists of a victim client and server on separate machines, both running GnuTLS and establishing an encrypted connection with static RSA key exchange under TLS 1.2. Further, an unprivileged attacker armed with DirtyRand is resident on the client.

- *Zap before seeding*: Prior to the client–server handshake—and before GnuTLS has seeded its PRNG—the attacker zaps the L1 DRNG, collapsing all kernel randomness on the client to a deterministic function of a known constant.
- *Recover the seed*: When GnuTLS keys its ChaCha20 PRNG it draws 32 random bytes from `getrandom()`, which the attacker computer offline. The 8-byte nonce—i.e., the second-resolution timestamp—used by the PRNG can be guessed by the attacker with a small margin of error if done close to the time of the handshake.
- *Reconstruct the PMS*: Feeding the recovered key and candidate nonces into a mirrored instance of the PRNG reproduces the 46 “random” bytes GnuTLS uses for the PMS.

Recovering the PMS suffices to break TLS as all other input used to generate the master secret—and thus session keys—travel in cleartext. As a result, after this exploit, the attacker can both decrypt and forge that session’s traffic.

6 RNGuard

6.1 Design

To mitigate DirtyRand, we present RNGuard: a protection mechanism for x86-64 and AArch64 that isolates the kernel RNG’s sensitive state. The attack in Section 4 establishes its impact under a deliberately weak adversary (§3); in contrast, our defense is engineered against a strictly stronger attacker with an arbitrary kernel read–write primitive. At a high level, we use Intel MPK on x86-64 and ARM MTE on AArch64 (§2.2) to confine access to RNG state to the narrow set of call sites in the kernel that legitimately require it—reads and writes from anywhere else in the kernel are blocked by hardware. The goal of our defense is not to improve the strength of the cryptographic algorithms used in the kernel; it is to make the small set of data structures the algorithms rely on inaccessible to anything outside the required RNG code paths.

A semi-automated audit of the Linux RNG (see Appendix G) identified three main, *persistent* data structures that require protection: the global L1 `base_crng`, the per-CPU L2 DRNGs, and the per-CPU L3 batch pools. For L1–L3, we protect each structure in its entirety by relocating them to a hardware-enforced, isolated region. Notably, this goes beyond protecting only the obviously sensitive key field (e.g., `base_crng.key`): the generation counter, the embedded locking primitives, and—for the batch pools—the position index are independently weaponizable, since an adversary who rolls generation backward or pins position

can force an RNG to prematurely reseed or replay previously-emitted output without ever touching the key, respectively.

One piece of attack-relevant state is deliberately left outside this boundary: the pending bit of the `crng_reseed` work item (§4.2). Halting the reseed cycle is useless without also corrupting or disclosing RNG state: the RNG will continue to perform ChaCha20 on still-secret data which provides an attacker with no useful foothold [22, 117, 119]. Additionally, given that corrupting or disclosing the data protected by RNGuard is a prerequisite for utilizing reseed halting, we did not want to bloat the kernel’s `workqueue` system. If protection of the pending bit is required in the future, we recommend using a lighter primitive, such as XOR-blinding the field [25, 43] with a per-boot secret held in the already-protected region.

The Linux RNG also exposes a plethora of *transient* state an attacker can (ab)use; e.g., on every random draw, the kernel allocates a fresh ChaCha20 state on the (readable and writable) stack and uses it for computation, creating an attacker-winnable race condition to predict the RNG’s output. Since this state is not a fixed allocation that can be relocated to a dedicated section at link time, it cannot be protected the way L1–L3 are. One option is to relocate the sensitive locals individually; this, however, leaves whatever the compiler elects to spill on an ordinary, attacker readable–writable stack. Instead, RNGuard moves the *execution context* that produces transient state into the protected domain: it maintains a pseudo-stack inside each protected region and switches the stack pointer to it for the duration of every RNG operation. Every spill and local the RNG produces therefore lands in memory the hardware mediates access to, with no per-variable auditing required. Both variants share this design and differ only in the primitive that isolates the pseudo-stack and in how the switch is performed (§6.1.1, §6.1.2).

6.1.1 MPK Variant

Isolation Primitive. To isolate memory on x86-64, we leverage MPK, (dis)allowing access to protected pages by tagging their PTEs with a 4-bit `pkey` that specifies corresponding access rights in the `PKRS MSR` (§2.2). We reserve only 1/16 `pkeys` for RNGuard—i.e., protected memory constitutes one trust domain—and initialize `PKRS[pkey]` to `0b11`, setting both the access-disable (`AD`) and write-disable (`WD`) bits so that reads and writes are denied even to supervisor code. Legitimate accesses to protected memory are bracketed by two gateway functions: one clears `PKRS[pkey]` to `0b00` before the access, the other restores it after. `PKRS` is per-logical-processor state rather than per-thread, so RNGuard holds interrupts off across the entire window: threads on other CPUs see their own `PKRS` with `AD/WD` still set, and on the issuing CPU no other thread or maskable handler executes while the gate is open. Attacker-controlled code therefore always runs with `AD/WD` set, and any direct attempt to read or write a protected page faults. Under the control-flow integrity assumption of

our threat model (§3), the attacker also cannot redirect control to call gateway functions from outside the RNG’s code paths.

Protecting Persistent State. To enforce page-level permissions with MPK without affecting adjacent kernel data, we isolate the sensitive, persistent RNG structures—L1, L2, L3—on their own pages via dedicated linker sections. Globally allocated state, such as `base_crng` (i.e., L1), contributes one additional page to the kernel’s `.data` section; each per-CPU region likewise gains one page for its share of the protected structures. Further, RNGuard unmaps all virtual pages in the kernel’s direct map (`physmap` [86]) that alias to the protected region. At boot, every PTE backing these sections is assigned our `pkey`, rendering L1–L3 unreachable outside the gateways.

Protecting Transient State. The section mechanism also supplies a home for transient state: we allocate a page-aligned, per-CPU *pseudo-stack* inside the isolated per-CPU region, where it inherits MPK protection. Each entry gateway then performs three steps rather than one: (1) disables interrupts, (2) clears `PKRS[pkey]`, and (3) switches `%rsp` to the executing CPU’s pseudo-stack. Exit gateways perform the inverse. Thus, while the RNG is executing, every spill and temporary value produced lands on protected pages, inaccessible to other concurrently running threads. Not only does this apply to the function directly proceeding an entry gateway, but all functions subsequently invoked. We allocate a single 4KB page for the pseudo-stack since the call graph following a gateway function is shallow and non-recursive; however, we still place an unmapped guard page after the pseudo-stack to deterministically detect overflows.

One RNG path cannot execute entirely inside a gateway. To accommodate the requirements of `copy_to_iter()`, which may fault, sleep, and reschedule, servicing a read of `/dev/urandom` must temporarily place chunks of random output in unprotected memory. RNGuard carefully limits this exposure by generating randomness in bounded chunks on the pseudo-stack and copies each chunk out afterwards.

6.1.2 MTE Variant

Isolation Primitive. To isolate memory on AArch64, we use MTE; every 16-byte granule of memory is associated with a 4-bit allocation tag, and the memory is made accessible by pointer if it carries a matching tag in its top byte (§2.2). Similar to the MPK scheme, we reserve a single tag value, representing a shared trust domain, and tag all isolated memory with it at boot. Notably, MTE is a probabilistic defense: with 4-bit tags, an attacker who forges a pointer has a 1/16 chance of guessing the tag correctly. What rescues this design from the usual fragility of probabilistic schemes is that *the attacker gets exactly one shot at guessing correctly*. An incorrect tag raises a synchronous tag-check fault that immediately panics the kernel. Combined with control-flow integrity (§3), which prevents the attacker from (ab)using the legitimate code paths in the kernel’s RNG that already hold correctly tagged point-

ers, the protected data is reachable only by gambling a kernel panic on a 1-in-16 guess—assuming the tag is not reserved statically (e.g., at compile time) and easily discoverable.

Protecting Persistent State. The memory-protection granule for MTE is much smaller than a MPK’s page granularity, so we do not need to relocate the persistent, sensitive RNG structures onto dedicated pages. The protected fields require only 16-byte alignment and can sit on the same pages as unrelated data. Significantly, MTE only checks tags on accesses made *through a pointer*, whereas the vanilla kernel reaches the RNG structures directly by name (e.g., `base_crng.key`). Thus, we re-route every access in the kernel’s RNG through dedicated, tagged pointers (e.g., `base_crng_p->key`).

Protecting Transient State. To maintain architectural parity with x86-64 and comprehensively safeguard transient data, RNGuard protects transient state on AArch64 by moving the execution context: it allocates a per-CPU pseudo-stack inside the isolated, tagged region, and the entry gateway pivots `sp` to it with the region’s tag in the pointer’s top byte. The tag is required rather than cosmetic—AArch64 exempts `sp-plus-immediate` accesses from tag checking, but not write-back forms or addresses the compiler materialises from `sp` into a general-purpose register. Consequently, every value the RNG computes, including compiler-introduced spills, is automatically hardware-isolated, with an unmapped guard page bounding overflow as on x86-64.

6.2 Implementation

We implemented our prototype of RNGuard for the Linux kernel v6.12.11 in $\approx 1.4\text{K}$ lines of C code, split evenly across the MPK and MTE variants. Our benchmarking machine (§7.2) did not have support for PKS, only PKU, which we repurposed to emulate the behavior of PKS. Specifically, we configured isolated data pages in the kernel’s address-space range as user pages; then, before accessing them from the kernel we toggled SMAP [65]. We kept these user-marked pages inaccessible to user space by unmapping them via PTI [67, 155].

7 Evaluation

7.1 Security Analysis

Our analysis of RNGuard’s security considers a capable, unprivileged, local adversary armed with an arbitrary kernel read-write primitive (§3). Importantly, the goal of RNGuard is not to improve the cryptographic strength of the kernel’s RNG—i.e., the design of ChaCha20 or BLAKE2s—but rather to fortify its implementation by strictly mediating access to its underlying state structures—e.g., the persistent state of L1–L3 and transient state. Further, we purposefully scope RNGuard to protect the RNG’s internal operation—randomness drawn from the RNG and used externally (e.g., `net_secret`; §5.2.2) is out of scope, left for other defenses and future work.

Persistent State Protection. To prevent unauthorized reads and writes of the RNG’s persistent structures, RNGuard leverages hardware-assisted memory isolation—Intel MPK on x86-64 and ARM MTE on AArch64 (§6.1). RNGuard relocates such sensitive structures into a dedicated, hardware-enforced trust domain only accessible by a narrow set of well-defined code paths (e.g., the MPK gateways that enable thread-specific access). Since we assume a CFI-hardened kernel (§3), an attacker is unable to arbitrarily steer execution to an access-granting code path; moreover, W^X prevents an adversary from injecting code that allows access. We also prevent attackers circumventing RNGuard through double-mappings in the kernel’s direct map (`physmap`) region, unmapping any aliases to the protected region. By defaulting the PKRS register of MPK to deny all access—or by utilizing an exclusive MTE allocation tag—RNGuard ensures that any attempt by an attacker to read or overwrite L1–L3 fields from outside an approved execution path will trigger a hardware fault, foiling their exploit. We determined that the L1–L3 structures are the only persistent state within scope requiring protection via data-flow analysis, described in Appendix G.

Transient State Protection. While isolating persistent structures prevents static state corruption and disclosure, the Linux RNG also exposes sensitive transient data, such as the ChaCha20 state initialized on the stack (§6.1). Because standard kernel stacks are readable and writable, an attacker can race the kernel, corrupting or disclosing this sensitive state during random number generation. RNGuard neutralizes this attack vector by isolating the execution context itself: upon entering the RNG, RNGuard pivots the stack pointer to a dedicated pseudo-stack located within the protected memory region. Thus, all stack spills, intermediate cryptographic state, and local variables produced during RNG operation are safely hardware-isolated, preventing unauthorized, race-condition reads and writes. We identified the functions requiring stack-switching via data-flow analysis (see Appendix G).

Case Study Exploit Neutralization. To empirically validate RNGuard’s security guarantees, we evaluate its effectiveness against our suite of end-to-end exploits presented in Section 5. In all scenarios, RNGuard successfully neutralizes the attacks by severing the adversary’s ability to manipulate or observe the underlying generator state—the foundational primitive for the case studies. By protecting the root of trust (the RNG), all dependent defense mechanisms—e.g., ASLR, stack canaries, TLS keys—retain assumed theoretical cryptographic security.

7.2 Performance Evaluation

Testbed. We evaluated the performance of RNGuard on a bare-metal x86-64 system equipped with an Intel Core i7-11800H (with MPK/PKU support) and 16GB of DDR4 RAM running Debian v12. We implemented our custom RNGuard kernel on Linux v6.12.11 and compared it against an unmodified v6.12.11 baseline, keeping PTI enabled for both. To

Table 2: LMBench overhead averaged over 10 runs.

Benchmark	Vanilla (μ s)	Ovh.	Benchmark	Vanilla (μ s)	Ovh.
System calls			Local IPC		
null syscall	0.405	0.1%	pipe	9.588	3.3%
read	0.503	0.1%	AF_UNIX	12.48	$\approx 0\%$
write	0.462	$\approx 0\%$	Process creation		
stat	1.171	0.9%	fork+exit	252.6	0.9%
fstat	0.627	$\approx 0\%$	fork+execve	595.4	$\approx 0\%$
open/close	2.075	$\approx 0\%$	fork+sh	1223.7	$\approx 0\%$
Signals & faults			Network latency (lo)		
sig. install	0.502	0.2%	UDP	14.38	1.8%
sig. handler	1.965	$\approx 0\%$	TCP	17.88	0.9%
prot. fault	0.999	0.1%	TCP connect	21.26	$\approx 0\%$
page fault	0.295	$\approx 0\%$	RPC/UDP	18.48	1.7%
select()			RPC/TCP	25.59	7.1%
10 fds	0.688	0.9%	Context switch (0k)		
100 fds	1.475	7.0%	2 procs	3.560	4.3%
250 fds	2.740	10.1%	4 procs	7.860	$\approx 0\%$
500 fds	5.018	9.6%	8 procs	7.086	2.4%
10 TCP fds	0.736	0.6%	16 procs	8.151	7.9%
100 TCP fds	3.275	0.5%	24 procs	8.251	9.0%
250 TCP fds	7.517	$\approx 0\%$	32 procs	7.978	2.7%
500 TCP fds	14.69	0.4%	64 procs	9.012	6.6%
			96 procs	9.250	5.3%

Table 3: Phoronix Test Suite per-test-family mean overhead.

Family	# Metrics	Ovh.	Family	# Metrics	Ovh.
pmbench	16	1.00%	OpenSSL	8	0.07%
OSBench	5	0.98%	Nginx	7	0.06%
Stress-NG	52	0.43%	Kernel build	2	0.01%
perf-bench	7	0.35%	Nettle	4	0.01%
GnuPG	1	0.30%	Loopback TCP	1	$\approx 0\%$
Redis	15	0.19%	WireGuard	1	$\approx 0\%$
SecureMark	1	0.16%			

minimize noise and ensure reproducibility, we ran all benchmarks in single-user mode, disabled DVFS, pinned the CPU frequency at 2GHz, and restricted C-states to C0. $\approx 0\%$ measurements denote $<0.01\%$ overhead.

LMBench. We first characterize RNGuard’s performance with LMBench v3.0-a9 [11], a suite of OS microbenchmarks that isolate individual kernel operations. Table 2 reports latencies averaged over 10 runs for both a vanilla kernel and an RNGuard-hardened kernel for x86-64. Out of the 36 tests performed by LMBench, 22 exhibit $<1\%$ overhead with 11 falling within measurement noise ($\approx 0\%$). Included in this set of 22 are the LMBench tests that cover the paths that extract randomness, and which RNGuard instruments most heavily. For example, `fork+execve` draws randomness during `fork()`, to create a kernel task canary (§5.1.1), and during `execve()` to fill `AT_RANDOM` (§5.1.1) and randomize the address-space layout (§5.1.2). Further, `TCP connect` draws an ephemeral source port and an ISN (§5.2.2). The results also reveal that overhead does not noticeably scale with number of draws, confirming the low cost of our gateways: `fork+execve` performs strictly more draws than `fork+exit`, but measures $\approx 0\%$. The 14 LMBench tests above 1% overhead fall in four families: `select()` on plain file descriptors ($\leq 10.1\%$), `pipe` (3.3%), `UDP` and `RPC` network latency (1.7–7.1%), and context switching ($\leq 9\%$). These paths perform few or no random draws, so the overhead stems primarily from RNGuard splitting large (2MB/1GB) pages into 4KB pages for `pkeys` and unmapping their `physmap` aliases.

Phoronix Test Suite. To assess RNGuard under realistic, real-world workloads, we ran the Phoronix Test Suite v10.8.4 [126], comparing a wide range of metrics across 13 test families spanning cryptography, networking, memory management, process and thread management, and general system stress. Table 3 shows the overhead of an RNGuard-hardened, x86-64 kernel atop an instrumented, vanilla kernel.

The tests one might expect to suffer the most under RNGuard are among the least affected: `OpenSSL` (0.07%), `Nettle` (0.01%), `SecureMark` (0.16%), `GnuPG` (0.3%), and `WireGuard` ($\approx 0\%$). However, this is consistent with how cryptographic consumers of randomness in user-space operate: libraries seed a user-space PRNG once from `getrandom()` and then generate a random stream locally. Thus, these programs may rarely draw from the kernel RNG, despite performing a lot of cryptographic work. This is also why `DirtyRand` is effective against them (§5.2.3): a single predictable seed compromises every subsequent output. At the other end, `osbench` (0.98%) and `pmbench` (1%) exhibit the highest overhead, exercising RNG-heavy paths like process creation.

8 Conclusion

We presented `DirtyRand`, an attack demonstrating that the internal state of the Linux kernel’s RNG is a highly vulnerable target for data-only attacks. Using only a weak, restricted-write primitive against the base DRNG state, an unprivileged local adversary can deterministically predict downstream randomness in both user and kernel space, collapsing the foundation numerous security mechanisms are built upon: we bypass stack canaries, ASLR, and freelist hardening, and mount DNS cache poisoning, TCP hijacking, and TLS pre-master secret recovery. We then introduced RNGuard, a defense that applies hardware-assisted memory isolation—Intel MPK on x86-64 and ARM MTE on AArch64—to the RNG’s persistent and transient state. RNGuard neutralizes these attacks even against a strictly stronger adversary with arbitrary read-write capability, at $\leq 1\%$ overhead on the Phoronix Test Suite.

Ethical Considerations

The research presented in this work involves the development and demonstration of exploitation techniques against the Linux kernel RNG. To ensure responsible research practices, all end-to-end exploits were developed and evaluated within isolated, locally hosted virtual machines. We did not target, exploit, or interact with any production systems, public networks, or third-party infrastructure.

The memory-corruption primitive used to demonstrate `DirtyRand` in our case studies is based on a publicly known and previously patched vulnerability (CVE-2022-34918), which we intentionally forward-ported to our test environment. Our objective in demonstrating `DirtyRand` is to investigate/-

study fundamental structural weaknesses in how OSES manage sensitive cryptographic state; our aim is not to introduce or weaponize new memory-corruption vulnerabilities. By presenting RNGuard alongside DirtyRand we provide a robust, hardware-backed mitigation to secure RNG state against current and future data-only attacks.

Open Science

In order to foster open science, we have publicly released all artifacts associated with this paper, including:

- The DirtyRand user-space library to predict RNG output.
- The DirtyRand case-study exploits, including payloads, kernel modules, and network scripts.
- The source code for RNGuard, including the MPK and MTE implementations ported to Linux v6.12.11.
- The configuration files and scripts used to execute the LM-Bench and Phoronix Test Suite performance evaluations.

The complete collection artifacts can be accessed at: anonymous.4open.science/r/dirtyrand-737D

References

- [1] CVE-2016-5195. <https://nvd.nist.gov/vuln/detail/cve-2016-5195>, 2016.
- [2] CVE-2015-1817. <https://nvd.nist.gov/vuln/detail/cve-2015-1817>, 2017.
- [3] CVE-2017-8824. <https://nvd.nist.gov/vuln/detail/cve-2017-8824>, 2017.
- [4] CVE-2022-0847. <https://nvd.nist.gov/vuln/detail/cve-2022-0847>, 2022.
- [5] CVE-2023-4911. <https://nvd.nist.gov/vuln/detail/cve-2023-4911>, 2023.
- [6] musl libc. <https://musl.libc.org>, 2023.
- [7] iputils. <https://github.com/iputils/iputils>, 2026.
- [8] Martín Abadi, Mihai Budiu, Ulfar Erlingsson, and Jay Ligatti. Control-flow integrity principles, implementations, and applications. *ACM Transactions on Information and System Security (TISSEC)*, 13(1):1–40, 2009.
- [9] Periklis Akrividis, Cristian Cadar, Costin Raiciu, Manuel Costa, and Miguel Castro. Preventing memory error exploits with wit. In *2008 IEEE Symposium on Security and Privacy (sp 2008)*, pages 263–277. IEEE, 2008.
- [10] Lars Ole Andersen. *Program Analysis and Specialization for the C Programming Language*. Phd thesis, University of Copenhagen, 1994.
- [11] Arm Ltd. Armv8.5-A Memory Tagging Extension. https://developer.arm.com/-/media/Arm%20Developer%20Community/PDF/Arm_Memory_Tagging_Extension_Whitepaper.pdf, 2019.
- [12] Jean-Philippe Aumasson and Daniel J Bernstein. SipHash: a fast short-input PRF. In *International Conference on Cryptology in India*, pages 489–508. Springer, 2012.
- [13] Jean-Philippe Aumasson, Samuel Neves, Zooko Wilcox-O’Hearn, and Christian Winnerlein. Blake2: simpler, smaller, fast as md5. In *International Conference on Applied Cryptography and Network Security*, pages 119–135. Springer, 2013.
- [14] Luigi Auriemma. NullLogic Null HTTPd 0.5.1 — Error Page Long HTTP Request Cross-Site Scripting. <https://www.exploit-db.com/exploits/23176>, 2003.
- [15] Pablo Neira Ayuso. netfilter: nf_tables: stricter validation of element data. Linux kernel commit 7e6bc1f6cabc, 2022. URL: <https://git.kernel.org/pub/scm/linux/kernel/git/torvalds/linux.git/commit/?id=7e6bc1f6cabcd30aba0b11219d8e01b952eacbb6>.
- [16] Michael Backes, Thorsten Holz, Benjamin Kollenda, Philipp Koppe, Stefan Nürnberger, and Jannik Pewny. You can run but you can’t read: Preventing disclosure exploits in executable code. In *Proceedings of the 2014 ACM SIGSAC Conference on Computer and Communications Security*, pages 1342–1353, 2014.
- [17] Arati Baliga, Pandurang Kamat, and Liviu Iftode. Lurking in the shadows: Identifying systemic threats to kernel data. In *2007 IEEE Symposium on Security and Privacy (SP’07)*, pages 246–251. IEEE, 2007.
- [18] Boaz Barak and Shai Halevi. A model and architecture for pseudo-random generation with applications to /dev/random. In *Proceedings of the 12th ACM conference on Computer and communications security*, pages 203–212, 2005.
- [19] Brian Belleville, Hyungon Moon, Jangseop Shin, Dongil Hwang, Joseph M Nash, Seonhwa Jung, Yeoul Na, Stijn Volckaert, Per Larsen, Yunheung Paek, et al. Hardware assisted randomization of data. In *International Symposium on Research in Attacks, Intrusions, and Defenses*, pages 337–358. Springer, 2018.

- [20] Steven M Bellovin. Security Problems in the TCP/IP Protocol Suite. *ACM SIGCOMM Computer Communication Review*, 19(2):32–48, 1989.
- [21] Steven M Bellovin. Using the Domain Name System for System Break-ins. In *USENIX Security Symposium (SEC)*, 1995.
- [22] Daniel J Bernstein. Fast-key-erasure random-number generators, 2017.
- [23] Daniel J Bernstein et al. Chacha, a variant of salsa20. In *Workshop record of SASC*, volume 8, pages 3–5. Lausanne, Switzerland, 2008.
- [24] Sandeep Bhatkar, Daniel C DuVarney, and Ron Sekar. Address obfuscation: An efficient approach to combat a broad range of memory error exploits. In *12th USENIX Security Symposium (USENIX Security 03)*, 2003.
- [25] Sandeep Bhatkar and R Sekar. Data space randomization. In *International Conference on Detection of Intrusions and Malware, and Vulnerability Assessment*, pages 1–22. Springer, 2008.
- [26] David Bigelow, Thomas Hobson, Robert Rudd, William Streilein, and Hamed Okhravi. Timely rerandomization for mitigating memory disclosures. In *Proceedings of the 22nd ACM SIGSAC Conference on Computer and Communications Security*, pages 268–279, 2015.
- [27] Andrea Bittau, Adam Belay, Ali Mashtizadeh, David Mazières, and Dan Boneh. Hacking blind. In *IEEE Symposium on Security and Privacy (S&P)*, pages 227–242. IEEE, 2014.
- [28] Tyler Bletsch, Xuxian Jiang, Vince W Freeh, and Zhenkai Liang. Jump-Oriented Programming: A New Class of Code-Reuse Attack. In *ACM Asia Symposium on Information, Computer and Communications Security (ASIACCS)*, pages 30–40, 2011.
- [29] Daniel Borkmann. bpf: add generic constant blinding for use in jits. Linux kernel commit 4f3446bb809f, 2016. URL: <https://git.kernel.org/pub/scm/linux/kernel/git/torvalds/linux.git/commit/?id=4f3446bb809f20ad56cadf712e6006815ae7a8f9>.
- [30] Mark Brand. MTE As Implemented, Part 1: Implementation Testing. <https://projectzero.google/2023/08/mte-as-implemented-part-1.html>, 2023.
- [31] Claudio Canella, Michael Schwarz, Martin Haubenwallner, Martin Schwarzl, and Daniel Gruss. Kaslr: Break it, fix it, repeat. In *ACM Asia Conference on Computer and Communications Security (AsiaCCS)*, pages 481–493, 2020.
- [32] Canella, Claudio and Genkin, Daniel and Giner, Lukas and Gruss, Daniel and Lipp, Moritz and Minkin, Marina and Moghimi, Daniel and Piessens, Frank and Schwarz, Michael and Sunar, Berk and Van Bulck, Jo, and Yarom, Yuval. Fallout: Leaking Data on Meltdown-resistant CPUs. In *ACM SIGSAC Conference on Computer and Communications Security (CCS)*, pages 769–784, 2019.
- [33] Canonical Ltd. DNS Security Extensions (DNSSEC). Ubuntu Server documentation, 2026. URL: <https://ubuntu.com/server/docs/explanation/dnssec/dnssec/>.
- [34] Nicholas Carlini, Antonio Barresi, Mathias Payer, David Wagner, and Thomas R Gross. {Control-Flow} bending: On the effectiveness of {Control-Flow} integrity. In *24th USENIX Security Symposium (USENIX Security 15)*, pages 161–176, 2015.
- [35] Scott A Carr and Mathias Payer. Datashield: Configurable data confidentiality and integrity. In *Proceedings of the 2017 ACM on Asia Conference on Computer and Communications Security*, pages 193–204, 2017.
- [36] Miguel Castro, Manuel Costa, and Tim Harris. Securing software by enforcing data-flow integrity. In *Proceedings of the 7th symposium on Operating systems design and implementation*, pages 147–160, 2006.
- [37] Stephen Checkoway, Lucas Davi, Alexandra Dmitrienko, Ahmad-Reza Sadeghi, Hovav Shacham, and Marcel Winandy. Return-Oriented Programming without Returns. In *ACM Conference on Computer and Communications Security (CCS)*, pages 559–572, 2010.
- [38] Quan Chen, Ahmed M Azab, Guruprasad Ganesh, and Peng Ning. Privwatcher: Non-bypassable monitoring and protection of process credentials from memory corruption attacks. In *Proceedings of the 2017 ACM on Asia Conference on Computer and Communications Security*, pages 167–178, 2017.
- [39] Shuo Chen, Jun Xu, Emre Can Sezer, Prachi Gauriar, and Ravishankar K Iyer. Non-control-data attacks are realistic threats. In *USENIX security symposium*, volume 5, page 146, 2005.
- [40] Xiangdong Chen, Zhaofeng Li, Tirth Jain, Vikram Narayanan, and Anton Burtsev. Limitations and opportunities of modern hardware isolation mechanisms. In *2024 USENIX Annual Technical Conference (USENIX ATC 24)*, pages 349–368, 2024.
- [41] Long Cheng, Salman Ahmed, Hans Liljestrand, Thomas Nyman, Haipeng Cai, Trent Jaeger, N Asokan,

- and Danfeng Yao. Exploitation techniques for data-oriented attacks with existing and potential defense approaches. *ACM Transactions on Privacy and Security (TOPS)*, 24(4):1–36, 2021.
- [42] Shaanan Cohney, Andrew Kwong, Shahar Paz, Daniel Genkin, Nadia Heninger, Eyal Ronen, and Yuval Yarom. Pseudorandom black swans: Cache attacks on `ctr_drbg`. In *2020 IEEE Symposium on Security and Privacy (SP)*, pages 1241–1258. IEEE, 2020.
- [43] Kees Cook. [[v4] mm: Add SLUB free list pointer obfuscation. Technical report, 2017. URL: <https://lkml.org/lkml/2017/7/26/4>.
- [44] D. Cooper, S. Santesson, S. Farrell, S. Boeyen, R. Housley, and W. Polk. Internet X.509 Public Key Infrastructure Certificate and Certificate Revocation List (CRL) Profile. RFC 5280, May 2008. URL: <https://www.rfc-editor.org/rfc/rfc5280>, doi: 10.17487/RFC5280.
- [45] Jonathan Corbet. Supervisor mode access prevention. LWN.net, 2012. URL: <https://lwn.net/Articles/517475/>.
- [46] Henry Corrigan-Gibbs, Wendy Mu, Dan Boneh, and Bryan Ford. Ensuring high-quality randomness in cryptographic key generation. In *Proceedings of the 2013 ACM SIGSAC conference on Computer & communications security*, pages 685–696, 2013.
- [47] Crispian Cowan, Calton Pu, Dave Maier, Jonathan Walpole, Peat Bakke, Steve Beattie, Aaron Grier, Perry Wagle, Qian Zhang, and Heather Hinton. Stackguard: Automatic adaptive detection and prevention of buffer-overflow attacks. In *USENIX Security Symposium (SEC)*, volume 98, pages 63–78. San Antonio, TX, 1998.
- [48] Crispin Cowan, Steve Beattie, John Johansen, and Perry Wagle. {PointGuard™}: Protecting pointers from buffer overflow vulnerabilities. In *12th USENIX Security Symposium (USENIX Security 03)*, 2003.
- [49] Stephen Crane, Christopher Liebchen, Andrei Homescu, Lucas Davi, Per Larsen, Ahmad-Reza Sadeghi, Stefan Brunthaler, and Michael Franz. Readactor: Practical code randomization resilient to memory disclosure. In *2015 IEEE Symposium on Security and Privacy*, pages 763–780. IEEE, 2015.
- [50] Lucas Davi, David Gens, Christopher Liebchen, and Ahmad-Reza Sadeghi. Pt-rand: Practical mitigation of data-only attacks against page tables. In *NDSS*, 2017.
- [51] Yu Ding, Zhuo Peng, Yuanyuan Zhou, and Chao Zhang. Android low entropy demystified. In *2014 IEEE international conference on communications (ICC)*, pages 659–664. IEEE, 2014.
- [52] Yevgeniy Dodis, David Pointcheval, Sylvain Ruhault, Damien Vergniaud, and Daniel Wichs. Security analysis of pseudo-random number generators with input: `/dev/random` is not robust. In *Proceedings of the 2013 ACM SIGSAC conference on Computer & communications security*, pages 647–658, 2013.
- [53] Mingshu Du, Yuan Ma, Na Lv, Tianyu Chen, Shijie Jia, and Fangyu Zheng. An empirical study on the quality of entropy sources in linux random number generator. In *ICC 2022-IEEE International Conference on Communications*, pages 559–564. IEEE, 2022.
- [54] Zakir Durumeric, Frank Li, James Kasten, Johanna Amann, Jethro Beekman, Mathias Payer, Nicolas Weaver, David Adrian, Vern Paxson, Michael Bailey, et al. The matter of heartbleed. In *Proceedings of the 2014 conference on internet measurement conference*, pages 475–488, 2014.
- [55] D. Eastlake, J. Schiller, and S. Crocker. Rfc 4086: Randomness requirements for security. Technical report, 2005.
- [56] Jake Edge. Enhancing FineIBT. <https://lwn.net/Articles/1039633/>, 2025.
- [57] Ulfar Erlingsson, Martín Abadi, Michael Vrabie, Mihai Budiu, and George C Necula. Xfi: Software guards for system address spaces. In *Proceedings of the 7th symposium on Operating systems design and implementation*, pages 75–88, 2006.
- [58] Adam Everspaugh, Yan Zhai, Robert Jellinek, Thomas Ristenpart, and Michael Swift. Not-so-random numbers in virtualized linux and the whirlwind rng. In *2014 IEEE Symposium on Security and Privacy*, pages 559–574. IEEE, 2014.
- [59] Dmitry Evtyushkin, Dmitry Ponomarev, and Nael Abu-Ghazaleh. Jump over ASLR: Attacking branch predictors to bypass ASLR. In *Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, pages 1–13. IEEE, 2016.
- [60] Free Software Foundation, Inc. Tunables (The GNU C Library). https://sourceware.org/glibc/manual/1/latest/html_node/Tunables.html, 2026.
- [61] Alexander J. Gaidis, Joao Moreira, Ke Sun, Alyssa Milburn, Vaggelis Atlidakis, and Vasileios P. Kemerlis. FineIBT: Fine-grain Control-flow Enforcement with

- Indirect Branch Tracking. In *International Symposium on Research in Attacks, Intrusions and Defenses (RAID)*, 2023.
- [62] Thomas Garnier. mm: SLAB freelist randomization. Linux kernel commit `c7ce4f60ac19`, 2016. Followed by SLUB port `210e7a43fa90`, July 2016. URL: <https://git.kernel.org/pub/scm/linux/kernel/git/torvalds/linux.git/commit/?id=c7ce4f60ac199fb4e93b8ec419c509cb91002551>.
- [63] Cristiano Giuffrida, Anton Kuijsten, and Andrew S Tanenbaum. Enhanced operating system security through efficient and fine-grained address space randomization. In *21st USENIX Security Symposium (USENIX Security 12)*, pages 475–490, 2012.
- [64] Fernando Gont and Steven M. Bellovin. Defending against sequence number attacks. <https://www.rfc-editor.org/rfc/rfc6528>, 2012.
- [65] Spyridoula Gravani, Mohammad Hedayati, John Criswell, and Michael L Scott. Fast intra-kernel isolation and security with iskios. In *Proceedings of the 24th international symposium on research in attacks, intrusions and defenses*, pages 119–134, 2021.
- [66] GRIMM Cyber. New old bugs in the linux kernel, 2021. URL: <https://grimmcyber.com/new-old-bugs-in-the-linux-kernel/>.
- [67] Daniel Gruss, Moritz Lipp, Michael Schwarz, Richard Fellner, Cl  mentine Maurice, and Stefan Mangard. Kaslr is dead: long live kaslr. In *International Symposium on Engineering Secure Software and Systems*, pages 161–176. Springer, 2017.
- [68] Daniel Gruss, Cl  mentine Maurice, Anders Fogh, Moritz Lipp, and Stefan Mangard. Prefetch side-channel attacks: Bypassing smap and kernel aslr. In *ACM SIGSAC Conference on Computer and Communications Security (CCS)*, pages 368–379, 2016.
- [69] Zvi Gutterman, Benny Pinkas, and Tzachy Reinman. Analysis of the linux random number generator. In *2006 IEEE Symposium on Security and Privacy (S&P’06)*, pages 15–pp. IEEE, 2006.
- [70] Nadia Heninger, Zakir Durumeric, Eric Wustrow, and J Alex Halderman. Mining your ps and qs: Detection of widespread weak keys in network devices. In *21st USENIX Security Symposium (USENIX Security 12)*, pages 205–220, 2012.
- [71] Hong Hu, Zheng Leong Chua, Sendroiu Adrian, Prateek Saxena, and Zhenkai Liang. Automatic generation of {Data-Oriented} exploits. In *24th USENIX Security Symposium (USENIX Security 15)*, pages 177–192, 2015.
- [72] Hong Hu, Shweta Shinde, Sendroiu Adrian, Zheng Leong Chua, Prateek Saxena, and Zhenkai Liang. Data-oriented programming: On the expressiveness of non-control data attacks. In *2016 IEEE Symposium on Security and Privacy (SP)*, pages 969–986. IEEE, 2016.
- [73] A. Hubert and R. van Mook. Measures for Making DNS More Resilient against Forged Answers. RFC 5452, January 2009. URL: <https://www.rfc-editor.org/rfc/rfc5452>, doi:10.17487/RFC5452.
- [74] Ralf Hund, Carsten Willems, and Thorsten Holz. Practical Timing Side Channel Attacks Against Kernel Space ASLR. In *IEEE Symposium on Security and Privacy (S&P)*, pages 191–205. IEEE, 2013.
- [75] Intel Corporation. Intel digital random number generator (DRNG) software implementation guide. <https://www.intel.com/content/www/us/en/developer/articles/guide/intel-digital-random-number-generator-drng-software-implementation-guide.html>, 2018.
- [76] Intel Corporation. *Intel[®] 64 and IA-32 Architectures Software Developer’s Manual*, 2018. URL: <https://www.intel.com/content/www/us/en/developer/articles/technical/intel-sdm.html>.
- [77] Intel Corporation. *Intel 64 and IA-32 Architectures Software Developer’s Manual, Volume 3A: System Programming Guide*, 2026. Section 5.6.2, “Protection Keys”.
- [78] Kyriakos K Ispoglou, Bader AlBassam, Trent Jaeger, and Mathias Payer. Block oriented programming: Automating data-only attacks. In *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security*, pages 1868–1882, 2018.
- [79] Hyerean Jang, Taehun Kim, and Youngjoo Shin. Sysbumps: Exploiting speculative execution in system calls for breaking kaslr in macos for apple silicon. In *ACM Conference on Computer and Communications Security (CCS)*, pages 64–78, 2024.
- [80] Yeongjin Jang, Sangho Lee, and Taesoo Kim. Breaking Kernel Address Space Layout Randomization with Intel TSX. In *ACM SIGSAC Conference on Computer and Communications Security (CCS)*, pages 380–392, 2016.
- [81] Jayden R. `pipe_buffer` arbitrary read/write. <https://www.interruptlabs.co.uk/articles/pipe-buffer>, November 2022.

- [82] Seth Jenkins. Defeating KASLR by Doing Nothing at All. <https://projectzero.google/2025/11/defeating-kaslr-by-doing-nothing-at-all.html>, 2025.
- [83] Dan Kaminsky. Black Ops 2008: It’s the End of the Cache As We Know It. Black Hat USA, 2008.
- [84] David Kaplan, Sagi Kedmi, Roei Hay, and Avi Dayan. Attacking the linux {PRNG} on android: Weaknesses in seeding of entropic pools and low {Boot-Time} entropy. In *8th USENIX workshop on offensive technologies (WOOT 14)*, 2014.
- [85] Gaurav S Kc, Angelos D Keromytis, and Vassilis Prevelakis. Countering Code-Injection Attacks With Instruction-Set Randomization. In *ACM Conference on Computer and Communications Security (CCS)*, pages 272–280, 2003.
- [86] Vasileios P Kemerlis, Michalis Polychronakis, and Angelos D Keromytis. ret2dir: Rethinking kernel isolation. In *23rd USENIX Security Symposium (USENIX Security 14)*, pages 957–972, 2014.
- [87] Chongkyung Kil, Jinsuk Jun, Christopher Bookholt, Jun Xu, and Peng Ning. Address space layout permutation (aslp): Towards fine-grained randomization of commodity software. In *2006 22nd Annual Computer Security Applications Conference (ACSAC’06)*, pages 339–348. IEEE, 2006.
- [88] Tom Kilburn, R Bruce Payne, and David J Howarth. The atlas supervisor. In *Proceedings of the December 12-14, 1961, eastern joint computer conference: computers-key to total systems control*, pages 279–294, 1961.
- [89] Soo Hyeon Kim, Daewan Han, and Dong Hoon Lee. Predictability of android openssl’s pseudo random number generator. In *Proceedings of the 2013 ACM SIGSAC conference on Computer & communications security*, pages 659–668, 2013.
- [90] Suryeon Kim, Seungwon Shin, and Hyunwoo Choi. AVX-TSCHA: Leaking information through AVX extensions in commercial processors. *Computers & Security*, 134:103437, 2023.
- [91] Amit Klein. Cross layer attacks and how to use them (for dns cache poisoning, device tracking and more). In *2021 IEEE Symposium on Security and Privacy (SP)*, pages 1179–1196. IEEE, 2021.
- [92] Hyungjoon Koo, Yaohui Chen, Long Lu, Vasileios P Kemerlis, and Michalis Polychronakis. Compiler-assisted code randomization. In *2018 IEEE symposium on security and privacy (SP)*, pages 461–477. IEEE, 2018.
- [93] Jakob Koschel, Cristiano Giuffrida, Herbert Bos, and Kaveh Razavi. TagBleed: Breaking KASLR on the Isolated Kernel Address Space using Tagged TLBs. In *IEEE European Symposium on Security and Privacy (EuroS&P)*, pages 309–321. IEEE, 2020.
- [94] Rashmi Kumari, Mohsen Alimomeni, and Reihaneh Safavi-Naini. Performance analysis of linux rng in virtualized environments. In *Proceedings of the 2015 ACM Workshop on Cloud Computing Security Workshop*, pages 29–39, 2015.
- [95] Volodymyr Kuznetsov, László Szekeres, Mathias Payer, George Candea, R Sekar, and Dawn Song. Code-pointer integrity. In *The Continuing Arms Race: Code-Reuse Attacks and Defenses*, pages 81–116. 2018.
- [96] Patrick Lacharme, Andrea Rock, Vincent Strubel, and Marion Videau. The linux pseudorandom number generator revisited. 2012.
- [97] Ziyu Li, Jun He, and Sheng Chen. Research on random number prediction and security threats in embedded linux systems. In *2025 IEEE 8th Information Technology and Mechatronics Engineering Conference (ITOEC)*, volume 8, pages 861–867. IEEE, 2025.
- [98] Hans Liljestrand, Thomas Nyman, Kui Wang, Carlos Chinea Perez, Jan-Erik Ekberg, and N Asokan. {PAC} it up: Towards pointer integrity using {ARM} pointer authentication. In *28th USENIX Security Symposium (USENIX Security 19)*, pages 177–194, 2019.
- [99] Zhenpeng Lin, Yuhang Wu, and Xinyu Xing. Dirtycred: Escalating privilege in linux kernel. In *Proceedings of the 2022 ACM SIGSAC conference on computer and communications security*, pages 1963–1976, 2022.
- [100] Moritz Lipp, Daniel Gruss, and Michael Schwarz. AMD Prefetch Attacks through Power and Time. In *USENIX Security Symposium (SEC)*, pages 643–660, 2022.
- [101] Moritz Lipp, Andreas Kogler, David Oswald, Michael Schwarz, Catherine Easdon, Claudio Canella, and Daniel Gruss. PLATYPUS: Software-based Power Side-Channel Attacks on x86. In *IEEE Symposium on Security and Privacy (S&P)*, pages 355–371. IEEE, 2021.
- [102] Lipp, Moritz and Schwarz, Michael and Gruss, Daniel and Prescher, Thomas and Haas, Werner and Fogh, Anders and Horn, Jann and Mangard, Stefan and Kocher, Paul and Genkin, Daniel and Yarom, Yuval and Hamburg, Mike. Meltdown: Reading Kernel Memory from User Space. In *USENIX Security Symposium (SEC)*, pages 973–990, 2018.

- [103] William Liu, Joseph Ravichandran, and Mengjia Yan. Entrybleed: A universal kaslr bypass against kpti on linux. In *International Workshop on Hardware and Architectural Support for Security and Privacy (HASP)*, pages 10–18, 2023.
- [104] LWN.net. x86 NX support. <https://lwn.net/Articles/87814/>, 2004.
- [105] Lukas Maar, Florian Draschbacher, Lukas Lamster, and Stefan Mangard. {Defects-in-Depth}: Analyzing the integration of effective defenses against {One-Day} exploits in android kernels. In *33rd USENIX Security Symposium (USENIX Security 24)*, pages 4517–4534, 2024.
- [106] A Theodore Marketos and Simon W Moore. The frequency injection attack on ring-oscillator-based true random number generators. In *International Workshop on Cryptographic Hardware and Embedded Systems*, pages 317–331. Springer, 2009.
- [107] Nikos Mavrogiannopoulos. GnuTLS manual, 2026. URL: <https://www.gnutls.org/manual/gnutls.html>.
- [108] Stephen McCamant and Greg Morrisett. Evaluating sfi for a cisc architecture. In *USENIX Security Symposium*, volume 10, pages 209–224, 2006.
- [109] D. McGrew. An Interface and Algorithms for Authenticated Encryption. RFC 5116, 2008. URL: <https://www.rfc-editor.org/info/rfc5116>, doi:10.17487/RFC5116.
- [110] Derrick Paul McKee, Yianni Giannaris, Carolina Ortega, Howard E Shrobe, Mathias Payer, Hamed Okhravi, and Nathan Burow. Preventing kernel hacks with hakcs. In *NDSS*, pages 1–17, 2022.
- [111] Larry W McVoy, Carl Staelin, et al. Lmbench: Portable tools for performance analysis. In *USENIX ATC*, volume 96, 1996.
- [112] Microsoft Docs. Data Execution Prevention. <https://docs.microsoft.com/en-us/>, 2022.
- [113] P. Mockapetris. Domain Names - Concepts and Facilities. RFC 1034, November 1987. URL: <https://www.rfc-editor.org/rfc/rfc1034>, doi:10.17487/RFC1034.
- [114] P. Mockapetris. Domain Names - Implementation and Specification. RFC 1035, November 1987. URL: <https://www.rfc-editor.org/rfc/rfc1035>, doi:10.17487/RFC1035.
- [115] Niels Möller. Nettle: A low-level cryptographic library, 2026. URL: <https://www.lysator.liu.se/~nisse/nettle/nettle.html>.
- [116] Marius Momeu, Alexander J Gaidis, Jasper vd Heidt, and Vasileios P Kemerlis. Iubik: Isolating user bytes in commodity operating system kernels via memory tagging extensions. In *2025 IEEE Symposium on Security and Privacy (SP)*, pages 867–885. IEEE, 2025.
- [117] Stephan Müller. Documentation and analysis of the linux random number generator. Technical report, Federal Office for Information Security (BSI), 2023. URL: https://www.bsi.bund.de/SharedDocs/Downloads/EN/BSI/Publications/Studies/LinuxRNG/LinuxRNG_EN_V5_4.pdf.
- [118] Nikolenko, Vitaly. CVE-2016-6187: Exploiting Linux kernel heap off-by-one. <https://duasynt.com/blog/cve-2016-6187-heap-off-by-one-exploit>, 2016.
- [119] Y. Nir and A. Langley. RFC 8439: ChaCha20 and Poly1305 for IETF Protocols. RFC 8439, 2018. URL: <https://www.rfc-editor.org/info/rfc8439/>.
- [120] OpenBSD. i386 W^X. <https://marc.info/?l=openbsd-misc&m=105056000801065>, 2003.
- [121] Elliott I Organick. *Computer system organization: the B5700/B6700 series*. Academic Press, 2014.
- [122] PaX Team. Address space layout randomization (aslr). <https://pax.grsecurity.net/docs/aslr.txt>.
- [123] PaX Team. Address Space Layout Randomization. <https://pax.grsecurity.net/docs/aslr.txt>, 2003.
- [124] Peterlin, Maxime and Pettersson, Philip and Adamski, Alexandre and Radocea, Alex. Exploiting a single instruction race condition in binder, 2020. URL: <https://www.longterm.io/cve-2020-0423.html>.
- [125] Theofilos Petsios, Vasileios P Kemerlis, Michalis Pochronakis, and Angelos D Keromytis. Dynaguard: Armoring canary-based protections against brute-force attacks. In *Proceedings of the 31st Annual Computer Security Applications Conference*, pages 351–360, 2015.
- [126] Phoronix Media. Phoronix Test Suite. <https://www.phoronix-test-suite.com/>, 2026.
- [127] Victor Le Pochat, Tom Van Goethem, Samaneh Tajalizadehkhoob, Wouter Joosen, et al. Tranco: A Research-Oriented Top Sites Ranking Hardened Against Manipulation. *arXiv preprint arXiv:1806.01156*, 2018.

- [128] Sergej Proskurin, Marius Momeu, Seyedhamed Ghavamnia, Vasileios P Kemerlis, and Michalis Polychronakis. xmp: Selective memory protection for kernel and user space. In *2020 IEEE Symposium on Security and Privacy (SP)*, pages 563–577. IEEE, 2020.
- [129] Weizhong Qiang, Jiawei Yang, Hai Jin, and Xuanhua Shi. Privguard: Protecting sensitive kernel data from privilege escalation attacks. *IEEE Access*, 6:46584–46594, 2018.
- [130] Qualcomm Technologies, Inc. Pointer authentication on ARMv8.3: Design and analysis of the new software security instructions. <https://www.qualcomm.com/content/dam/qcomm-martech/dm-assets/documents/pointer-auth-v7.pdf>, 2017.
- [131] Qualys. Looney Tunables: Local Privilege Escalation in the glibc’s ld.so (CVE-2023-4911). <https://www.qualys.com/2023/10/03/cve-2023-4911/looney-tunables-local-privilege-escalation-glibc-ld-so.txt>, 2023.
- [132] Hany Ragab, Alyssa Milburn, Kaveh Razavi, Herbert Bos, and Cristiano Giuffrida. Crosstalk: Speculative data leaks across cores are real. In *2021 IEEE symposium on security and privacy (SP)*, pages 1852–1867. IEEE, 2021.
- [133] Prabhu Rajasekaran, Stephen Crane, David Gens, Yeoul Na, Stijn Volckaert, and Michael Franz. Codarr: Continuous data space randomization against data-only attacks. In *Proceedings of the 15th ACM Asia Conference on Computer and Communications Security*, pages 494–505, 2020.
- [134] RandoriSec. Crack Linux Firewall (CVE-2022-34918). <https://blog.randorisec.fr/fr/crack-linux-firewall/>, 2022.
- [135] E. Rescorla. The Transport Layer Security (TLS) Protocol Version 1.3. RFC 8446, August 2018. URL: <https://www.rfc-editor.org/rfc/rfc8446>, doi:10.17487/RFC8446.
- [136] E. Rescorla and T. Dierks. The Transport Layer Security (TLS) Protocol Version 1.2. RFC 5246, August 2008. URL: <https://www.rfc-editor.org/rfc/rfc5246>, doi:10.17487/RFC5246.
- [137] Ryan Riley. A framework for prototyping and testing data-only rootkit attacks. *Computers & security*, 37:62–71, 2013.
- [138] Thomas Ristenpart and Scott Yilek. When good randomness goes bad: Virtual machine reset vulnerabilities and hedging deployed cryptography. In *Ndss*, 2010.
- [139] Sandro Rügge, Johannes Wikner, and Kaveh Razavi. Branch Privilege Injection: Compromising Spectre v2 Hardware Mitigations by Exploiting Branch Predictor Race Conditions. In *USENIX Security Symposium (SEC)*, volume 25, 2025.
- [140] Muhammad Asad Saleem, Zahid Ghaffar, Khalid Mahmood, Ashok Kumar Das, Joel JPC Rodrigues, and Muhammad Khurram Khan. Provably secure authentication protocol for mobile clients in iot environment using puncturable pseudorandom function. *IEEE Internet of Things Journal*, 8(22):16613–16622, 2021.
- [141] Cole Schlesinger, Karthik Pattabiraman, Nikhil Swamy, David Walker, and Ben Zorn. Yarra: An extension to c for data integrity and partial safety, 2011.
- [142] Felix Schuster, Thomas Tendyck, Christopher Liebchen, Lucas Davi, Ahmad-Reza Sadeghi, and Thorsten Holz. Counterfeit Object-oriented Programming: On the Difficulty of Preventing Code Reuse Attacks in C++ Applications. In *IEEE Symposium on Security and Privacy (S&P)*, pages 745–762, 2015.
- [143] Hovav Shacham, Matthew Page, Ben Pfaff, Eu-Jin Goh, Nagendra Modadugu, and Dan Boneh. On the effectiveness of address-space randomization. In *Proceedings of the 11th ACM conference on Computer and communications security*, pages 298–307, 2004.
- [144] Chengyu Song, Byoungyoung Lee, Kangjie Lu, William Harris, Taesoo Kim, and Wenke Lee. Enforcing kernel security invariants with data flow integrity. In *NDSS*, 2016.
- [145] Chengyu Song, Hyungon Moon, Monjur Alam, Insu Yun, Byoungyoung Lee, Taesoo Kim, Wenke Lee, and Yunheung Paek. Hdfi: Hardware-assisted data-flow isolation. In *2016 IEEE Symposium on Security and Privacy (SP)*, pages 1–17. IEEE, 2016.
- [146] Mathilde Soucarros, Cécile Canovas-Dumas, Jessy Clédière, Philippe Elbaz-Vincent, and Denis Réal. Influence of the temperature on true random number generators. In *2011 IEEE International Symposium on Hardware-Oriented Security and Trust*, pages 24–27. IEEE, 2011.
- [147] Ana Nora Sovarel, David Evans, and Nathanael Paul. Where’s the feeb? the effectiveness of instruction set randomization. In *USENIX Security Symposium (SEC)*, volume 10, 2005.
- [148] Maddie Stone. Bad Binder: Android In-The-Wild Exploit. <https://projectzero.google/2019/11/bad-binder-android-in-wild-exploit.html>, 2019.

- [149] Yulei Sui and Jingling Xue. Svf: interprocedural static value-flow analysis in llvm. In *Proceedings of the 25th international conference on compiler construction*, pages 265–266, 2016.
- [150] Jiadong Sun, Xia Zhou, Wenbo Shen, Yajin Zhou, and Kui Ren. Pesc: A per system-call stack canary design for linux kernel. In *Proceedings of the Tenth ACM Conference on Data and Application Security and Privacy*, pages 365–375, 2020.
- [151] systemd project. resolved.conf, resolved.conf.d: Network name resolution configuration files. Linux manual pages, 2026. URL: <https://www.man7.org/linux/man-pages/man5/resolved.conf.5.html>.
- [152] systemd project. systemd-resolved.service, systemd-resolved: Network name resolution manager. Linux manual pages, 2026. URL: <https://www.man7.org/linux/man-pages/man8/systemd-resolved.service.8.html>.
- [153] László Szekeres, Mathias Payer, Tao Wei, and Dawn Song. SoK: Eternal war in memory. In *IEEE Symposium on Security and Privacy (S&P)*, pages 48–62. IEEE, 2013.
- [154] Adrian Tang, Simha Sethumadhavan, and Salvatore Stolfo. Heisenbyte: Thwarting memory disclosure attacks using destructive code reads. In *Proceedings of the 22nd ACM SIGSAC Conference on Computer and Communications Security*, pages 256–267, 2015.
- [155] The Linux Kernel Development Community. Page Table Isolation (PTI). <https://docs.kernel.org/arch/x86/pti.html>, 2026.
- [156] Caroline Tice, Tom Roeder, Peter Collingbourne, Stephen Checkoway, Úlfar Erlingsson, Luis Lozano, and Geoff Pike. Enforcing Forward-Edge Control-Flow Integrity in GCC & LLVM. In *USENIX Security Symposium (SEC)*, pages 941–955, 2014.
- [157] Ubuntu Security Team. Ubuntu Security Features. <https://wiki.ubuntu.com/Security/Features>, 2026.
- [158] Anjo Vahldiek-Oberwagner, Eslam Elnikety, Nuno O Duarte, Michael Sammler, Peter Druschel, and Deepak Garg. {ERIM}: Secure, efficient in-process isolation with protection keys ({MPK}). In *28th USENIX Security Symposium (USENIX Security 19)*, pages 1221–1238, 2019.
- [159] Victor van der Veen, Dennis Andriesse, Manolis Stamatiogiannakis, Xi Chen, Herbert Bos, and Cristiano Giuffrida. The Dynamics of Innocent Flesh on the Bone: Code Reuse Ten Years Later. In *ACM Conference on Computer and Communications Security (CCS)*, pages 1675–1689, 2017.
- [160] Victor Van der Veen, Nitish Dutt-Sharma, Lorenzo Cavallaro, and Herbert Bos. Memory errors: The past, the present, and the future. In *International Workshop on Recent Advances in Intrusion Detection*, pages 86–106. Springer, 2012.
- [161] Fernando Vano-Garcia and Hector Marco-Gisbert. E-boot: Preventing boot-time entropy starvation in cloud systems. *IEEE Access*, 8:61872–61890, 2020.
- [162] Robert Wahbe, Steven Lucco, Thomas E Anderson, and Susan L Graham. Efficient software-based fault isolation. In *Proceedings of the fourteenth ACM symposium on Operating systems principles*, pages 203–216, 1993.
- [163] Qian Wang, Hai Su, Kui Ren, and Kwangjo Kim. Fast and scalable secret key generation exploiting channel phase randomness in wireless networks. In *2011 Proceedings IEEE INFOCOM*, pages 1422–1430. IEEE, 2011.
- [164] Daniel Weber, Ahmad Ibrahim, Hamed Nemati, Michael Schwarz, and Christian Rossow. Osiris: Automated Discovery of Microarchitectural Side Channels. In *USENIX Security Symposium (SEC)*, pages 1415–1432, 2021.
- [165] David Williams-King, Graham Gobieski, Kent Williams-King, James P Blake, Xinhao Yuan, Patrick Colp, Michelle Zheng, Vasileios P Kemerlis, Junfeng Yang, and William Aiello. Shuffler: fast and deployable continuous code {re-randomization}. In *12th USENIX Symposium on Operating Systems Design and Implementation (OSDI 16)*, pages 367–382, 2016.
- [166] Wei Wu, Yueqi Chen, Jun Xu, Xinyu Xing, Xiaorui Gong, and Wei Zou. FUZE: Towards Facilitating Exploit Generation for Kernel Use-After-Free Vulnerabilities. In *USENIX Security Symposium (SEC)*, pages 781–797, 2018.
- [167] Shaohao Xie, Fangguo Zhang, and Rong Cheng. Security enhanced rfid authentication protocols for health-care environment. *Wireless Personal Communications*, 117(1):71–86, 2021.
- [168] Jun Xu, Zbigniew Kalbarczyk, and Ravishankar K Iyer. Transparent runtime randomization for security. In *22nd International Symposium on Reliable Distributed Systems, 2003. Proceedings.*, pages 260–269. IEEE, 2003.

- [169] Bennet Yee, David Sehr, Gregory Dardyk, J Bradley Chen, Robert Muth, Tavis Ormandy, Shiki Okasaka, Neha Narula, and Nicholas Fullagar. Native client: A sandbox for portable, untrusted x86 native code. *Communications of the ACM*, 53(1):91–99, 2010.
- [170] Kyle Zeng. [cve-2022-1786] a journey to the dawn, 2022. URL: <https://blog.kylebot.net/2022/10/16/CVE-2022-1786/>.
- [171] Kyle Zeng, Yueqi Chen, Haehyun Cho, Xinyu Xing, Adam Doupé, Yan Shoshitaishvili, and Tiffany Bao. Playing for K(H)eaps: Understanding and Improving Linux Kernel Exploit Reliability. In *USENIX Security Symposium (SEC)*, pages 71–88, 2022.
- [172] Jinmeng Zhou, Ziyue Pan, Jiayi Hu, Jiaxun Zhu, Wenbo Shen, Rui Chang, Guoren Li, and Zhiyun Qian. Beyond control: Exploring novel file system objects for data-only attacks on linux systems. *IEEE Transactions on Dependable and Secure Computing*, 2026.

A Data-only Attacks (Extended)

Software written in memory- and/or type-unsafe languages—such as C, C++, and assembly—is vulnerable to *memory errors* [153, 160], which allow attackers to corrupt or disclose data within a program’s address space. Historically, attackers targeted *control data*: return addresses [159], function pointers [28, 37], and dynamic dispatch tables [142]—collectively, *code pointers* [95]. Such *control-oriented attacks hijack* the program’s control flow to achieve arbitrary code execution. However, the widespread deployment of modern exploit mitigations has significantly increased the cost of these attacks. Mitigation examples include fine-grained address space randomization [63, 87, 92] and re-randomization [26, 165], memory leakage prevention [16, 49, 154], code-pointer integrity [95], and control-flow integrity [8, 61, 156].

In response, attackers have instead shifted toward *data-only* attacks, which manipulate *non-control data* to influence program behavior while remaining within legitimate control-flow paths [1, 4, 14, 34, 39, 41, 54, 71, 72, 78, 99, 137, 148]. Because code pointers are not corrupted to hijack control flow, these attacks typically circumvent the mitigations listed above. The Linux kernel has been a particularly fruitful target, for example: *Dirty COW* [1] enables privilege escalation by racing the kernel’s copy-on-write (COW) mechanism to write to read-only mappings; *DirtyPipe* [4] exploits stale pipe buffer flags to overwrite page-cache contents backed by read-only files; and *DirtyCred* [99] swaps unprivileged kernel credential structures with privileged ones to achieve privilege escalation without overwriting code pointers. Similar exploit techniques target kernel objects such as `modprobe_path`—i.e., a kernel-resident string that specifies a user-space program for (un)installing kernel modules, which can be overwritten to

point to an attacker program—and page-table entries [50]. Separately, information disclosure vulnerabilities such as *Heartbleed* [54] demonstrate that non-control data itself—e.g., cryptographic keys, session state, and credentials—are security-critical assets even before corruption.

Research has further demonstrated that data-only attacks are capable of expressive logic, producing the same effects as control-flow hijacking attacks [34, 71, 78] and achieving Turing-complete behavior [72]. *Data-oriented programming* (DOP) [72] chains operations together through corruption of non-control data to create expressive data-only attacks, while *block-oriented programming* [78] chains basic blocks to stay within valid paths. Automatic discovery and creation of such chains has also been demonstrated [34, 71, 78].

Defending against data-only attacks is fundamentally harder than defending against control-flow hijacking. A program’s execution path is a “rigid,” mathematical structure—i.e., its control-flow graph (CFG)—that mitigations can be built against with low overhead. For example, CFI [8] constrains indirect branches to a statically computed CFG. In contrast, data is inherently fluid, dynamically changing according to program behavior and external input. Since the CPU lacks the semantic context to determine *why* data is changing, it cannot easily differentiate if the update is malicious.

To address this, prior work has explored several defenses against data-only attacks, typically trading cost for coverage. Many of these defenses broadly seek to ensure that data objects are modified only by authorized code. Examples include data-flow integrity (DFI), which constrains runtime data-flow according to a statically computed data-flow graph [36], its hardware-assisted variant HDFI [145], and Write Integrity Testing (WIT), which restricts writes to legal sets of write instructions and target objects [9]. Orthogonally, data space randomization (DSR) masks data objects in memory so that an attacker corrupting memory cannot predict or control their values [19, 25, 48, 133]. Other defenses, such as DataShield [35] and Yarra [141], extend memory-unsafe languages such as C and C++ with enforcement mechanisms that allow developers to declare and protect security-critical data objects. A complementary line of work focuses on protecting specific security-critical objects, particularly those whose corruption directly enables privilege escalation. Examples of such work includes PrivWatcher [38], PrivGuard [129], and IU-BIK [116]. Another line of work employs memory isolation and hardware-assisted access control to *isolate* sensitive data. These approaches leverage hardware mechanisms such as Intel Memory Protection Keys (MPK) [76] and ARM Memory Tagging Extensions (MTE) [11] (§2.2). Systems such as ERIM [158], xMP [128], and HAKC [110] build on such primitives to protect sensitive application and kernel state.

However, these mechanisms assume that one can effectively determine *what* data should be classified as *security-critical*, and thus require protection. This problem is orthogonal to the enforcement mechanisms detailed above and

is typically resolved either manually [35, 141] or automatically [144, 172] with partial coverage.

B Attacks against the RNG (Extended)

The security of every randomness consumer across user and kernel space ultimately reduces to the unpredictability and integrity of the kernel’s RNG state. Prior work has repeatedly demonstrated that weaknesses in entropy collection, premature use of the generator before it is properly seeded, or insufficient mixing can propagate downstream into failures of higher-level mechanisms, ranging from cryptographic-key collisions across Internet-facing devices [70] to direct cryptanalysis of the Linux RNG [69, 84]. Consequently, the Linux RNG has long been treated as a high-value attack target.

Entropy Starvation. Entropy starvation during early boot presents a significant attack surface. During initialization, the RNG may not have accumulated sufficient entropy to provide strong randomness guarantees, allowing an attacker to reduce the search space of possible internal states. Kaplan et al. [84] demonstrated that low boot-time entropy, combined with leakage of RNG output, could enable attacks on the generator state, facilitating IPv6 fragmentation prediction and stack-canary bypasses. Heninger et al. [70] similarly found that a boot-time entropy hole could cause the generation of weak TLS and SSH keys. Related work has also examined boot-time entropy starvation in embedded Linux systems [97]. Low-entropy initialization has also affected Android systems, where security mechanisms such as stack canaries and ASLR relied on randomness produced before the RNG was fully initialized [51]; the Android OpenSSL RNG similarly suffered from a boot-time entropy hole that enabled recovery of the `PreMasterSecret` for the first SSL session [89]. Similar concerns arise in virtual environments, where low entropy at VM boot [58] or VM state reuse [138] can expose cryptographic keys generated after snapshot resumption. More recent analyses indicate that entropy starvation affects user-space applications and kernel functions during early boot [161].

Entropy Degradation. Beyond early-boot starvation, researchers have examined the quality of the entropy sources themselves; in several cases, sources contribute substantially less unpredictability than the kernel’s estimator credits them with. Du et al. [53] found that the Linux entropy estimator overestimates the contribution of jiffies, which are highly predictable to an attacker with access to a high-resolution cycle counter. Prior work has also shown that an attacker may artificially manipulate entropy sources via frequency injection [106] or temperature control [146] to substantially degrade collected entropy. Some attacks target the contents of the entropy pools themselves, repeatedly corrupting both the collected entropy and the associated mixing functions to degrade the quality of generated randomness [17]. Virtualization introduces its own entropy-quality problems: Kumari et al. [94] report that guests generate randomness at a lower rate

than their hosts and can influence the host’s entropy pool, reducing the unpredictability of RNG outputs system-wide.

State Compromise. The security of the RNG depends on its forward- and backward-security guarantees, which protect past and future random outputs, respectively, despite state exposure [18]. Historically, Gutterman et al. [69] demonstrated that in earlier iterations of the Linux RNG, given a snapshot of the RNG state, it is possible to reconstruct previous generator states and outputs, violating forward-security guarantees; subsequent changes to the RNG design addressed these weaknesses [96]. However, Dodis et al. [52] showed that under slow entropy accumulation, neither `/dev/random` nor `/dev/urandom` guarantees recovery following state compromise due to flaws in the entropy estimator and fixing function, violating backward-security guarantees. Beyond Linux, Cohn et al. [42] recently demonstrated that a cache attack recovering only part of the NIST CTR_DRBG state (used in OpenSSL and NetBSD) is enough to predict future outputs.

C Linux Kernel RNG Structures

The L1 base DRNG is declared in the Linux kernel (in `drivers/char/random.c`) as follows:

```
struct base_crng {
    u8          key[CHACHA_KEY_SIZE];
    unsigned long generation;
    spinlock_t   lock;
};
```

A representative example of an L3 per-CPU batch pool declaration (from `drivers/char/random.c`) is shown below, using `u32` as the integer type:

```
struct batch_u32 {
    u32          entropy[24]; /* 96B */
    local_lock_t lock;
    unsigned long generation;
    unsigned int  position;
};
```

D Determining Workqueue Target Addresses

A given workqueue is associated with a worker pool via a *pool workqueue* (PWQ), an interface that represents a specific workqueue–worker pool pairing. When a work item is queued, the kernel selects a PWQ to manage execution. Because a workqueue may be serviced by multiple worker pools, a single work item can be associated with multiple PWQs.

A key challenge in halting the reseed cycle is that the target data—the reseed work item’s pending bit—cannot be modified independently, as it is embedded within the structure’s data, which encodes multiple flags. Partial overwrites leave the field in an inconsistent state. Instead, the attack must overwrite the *entire* field with a semantically valid value.

Configuration	RAM Size	Base Address	Offsets	Total
n	1 GB	0xffff888003876000	{0x200, 0x400, 0x600, 0x800}	4
n	2 GB	0xffff888003876000	{0x200, 0x400, 0x600, 0x800}	4
n	4 GB	0xffff888100076000	{0x200, 0x400, 0x600, 0x800}	4
n	8 GB	0xffff888100076000	{0x200, 0x400, 0x600, 0x800}	4
n	16 GB	0xffff888100076000	{0x200, 0x400, 0x600, 0x800}	4
y	1 GB	0xffff888003870000	{0xe000 + 0x200 · i i ∈ [0, 15]}	16
y	2 GB	0xffff888003870000	{0xe000 + 0x200 · i i ∈ [0, 15]}	16
y	4 GB	0xffff888100070000	{0xe000 + 0x200 · i i ∈ [0, 15]}	16
y	8 GB	0xffff888100070000	{0xe000 + 0x200 · i i ∈ [0, 15]}	16
y	16 GB	0xffff888100070000	{0xe000 + 0x200 · i i ∈ [0, 15]}	16

Table 4: Comparison of PWQ address patterns with `SLAB_FREELIST_RANDOM` disabled (“n”) versus enabled (“y”) across different RAM sizes. Addresses are expressed as `base + offset`.

In particular, determining a valid payload requires identifying a valid pool workqueue (PWQ) address. PWQ structures are allocated early in the kernel boot process during `workqueue_init_early`. As these allocations occur while only minimal subsystems are available, the locations of PWQ structures are highly stable across boots.

To evaluate address stability, we measure the set of PWQ addresses associated with the `crng_reseed` work item across different kernel configurations and system memory sizes. Because a work item may execute on any eligible worker pool, multiple PWQ addresses can be observed across a single boot. In practice, the work item consistently cycles among a small number of PWQs.

We evaluate the effects of `CONFIG_SLAB_FREELIST_RANDOM`, `CONFIG_SHUFFLE_PAGE_ALLOCATOR`, and the `page_alloc.shuffle` boot parameter on systems with up to 16GB of RAM. We find that page allocator shuffling has no observable impact on PWQ placement. When `CONFIG_SLAB_FREELIST_RANDOM` is disabled, the `crng_reseed` work item is confined to four PWQ addresses. The PWQ address base differs between smaller (1GB, 2GB) and larger (4GB, 8GB, 16GB) memory configurations, but remains stable within each system memory size (Table 4).

Enabling `CONFIG_SLAB_FREELIST_RANDOM` increases the number of observed PWQ addresses but remains similarly predictable. Across all evaluated systems, valid PWQ addresses are drawn from a set of 16 fixed locations (Table 4). As with the non-randomized configuration, the PWQ address base is stable for a given system memory size and independent of page allocator shuffling.

E Freelist Hardening Case Study (Extended)

The Linux kernel’s SLUB allocator services dynamic memory requests for kernel objects from caches of equally sized objects, each backed by *slabs*: contiguous collections of pages carved into object slots. The free slots of a slab are tracked by a *freelist*, a singly-linked list whose `next` pointers are stored

within the free objects themselves. Since every allocation is served by popping a freelist head, the integrity of these pointers directly determines the integrity of the kernel heap.

Attackers have long abused memory-corruption bugs—both out-of-bounds writes [66, 118] and use-after-frees [124, 170]—to overwrite the freelist pointer of a freed SLUB object. A corrupted pointer causes the allocator to return an attacker-chosen address on a subsequent allocation, yielding an arbitrary-write primitive and, ultimately, privilege escalation. To mitigate such attacks, the kernel deploys *freelist pointer encoding* [43, 105], which stores freelist pointers obfuscated under a per-cache secret derived from a randomly generated value rather than in the clear. Without knowledge of that secret, an attacker cannot construct a pointer that decodes to a chosen address, so freelist corruption degrades from a reliable primitive into a blind guess.

Exploit Technique. To demonstrate DirtyRand’s efficacy against heap hardening, we build a cross-cache privilege-escalation exploit around CVE-2017-8824 [3], a use-after-free in the Linux kernel’s DCCP implementation. A DCCP socket’s disconnect path fails to release and clear its CCID pointers. If the socket is then reused as a listener, newly accepted child sockets inherit these pointers. Destroying a child frees the shared `struct ccid` while the original socket retains a dangling pointer. We restrict ourselves to the two capabilities such a bug typically yields: (1) disclosure of the dangling `struct ccid` address, and (2) a constrained read-write confined to the first 8 bytes at that address. Recovering an object address and reclaiming the freed slot to obtain a restricted write are the standard products of kernel use-after-free exploitation [171], and prior work derives them semi-automatically [166]. Importantly, we assume no arbitrary read and no arbitrary write.

Our write can only corrupt a freelist pointer inside a freed `ccid` slot, but the target—`struct cred`—lives in a separate cache, `cred_jar`. As is typical in cross-cache attacks, we bridge the two at the page level. After opening enough DCCP connections to spray several thousand `ccid` objects, we free

the non-anchor, client-side CCIDs through the CVE trigger and close their server-side peers to ensure one page stays partially occupied while the rest are freed and returned to the buddy allocator. We create child processes before the drain and wake them afterward to invoke `capset()`. Each `capset()` invocation forces `prepare_creds()` to allocate fresh credentials, driving `cred_jar` to request pages that were just returned to the buddy allocator. A successful spray attempt leaves a DCCP socket’s dangling CCID pointer aliased with a live credential object.

We then use the constrained read as an oracle to discover and differentiate free memory slots and freshly allocated credentials. Specifically, by reading the first 8 bytes from our collection of dangling pointers we determine whether the slot holds an encoded freelist pointer—a large, effectively random value—or a credential reference count—a small value. By using capability (1) we can also determine the address of the aliased credential structure.

The `struct ccid` allocator is then hijacked to serve a fresh `ccid` object aliased to the live credential we previously identified. We free one `ccid` anchor structure so that its slot becomes the second freelist entry (without triggering page reclamation; i.e., we still have other anchor structures allocated). Next, we forge the freelist with two constrained writes: we overwrite the anchor slot’s first 8 bytes with an encoded pointer to the target credential object, and we overwrite the credential’s first 8 bytes with the encoded representation of `null`, terminating the freelist. Importantly, the latter write lands on `cred->usage` so no privilege-related field is ever overwritten by this phase of the exploit. Because the `ccid` cache is created *after* the RNG is compromised, DirtyRand derives the cache’s freelist secret deterministically and both forged links decode legally.

Finally, by making new DCCP connections, fresh `ccid` objects are allocated from our forged freelist resulting in the target credential object being used as a `ccid` object in the DCCP connection procedure. As a part of this connection procedure, the new `ccid` objects are initialized, overwriting `cred->usage` with the CCID operations pointer and setting `cred->uid` to 0. The kernel, not the exploit, performs the privilege-granting write. The spray child owning that credential observes `getuid() == 0`, calls `setresuid()`, and executes a root shell.

F ASLR Offset Calculations

Table 5 shows the calculations performed for randomizing different address-space regions, in addition to stack jitter, for Linux v6.12.11 on x86-64.

Table 5: ASLR calculations in Linux (v6.12.11) for x86-64. Each row gives the offset added to a fixed region base and the entropy in bits. R_{64} and R_{32} denote 64-bit and 32-bit values returned from `get_random_long()` and `get_random_u32`, respectively. $P=2^{12}$ is a 4KB page size.

Region	Offset	Bits
Stack top	$(R_{64} \bmod 2^{22})P$	22
Stack jitter	$-(R_{32} \bmod 2^{13}) \wedge \sim 0xf$	9
mmap base	$-(R_{64} \bmod 2^{28})P$	28
PIE load bias	$+(R_{64} \bmod 2^{28})P$	28
ET_DYN loader	<i>inherits mmap base</i>	28
brk heap	$+(R_{64} \bmod 2^{18})P$	18

Table 6: Kernel RNG APIs and the subsystem of their consumers. Counts are call sites. *Other* groups process creation, UUID generation, RNG core functionality, and more.

RNG API	Net	Mem	FS	Crypto	Gfx	Other	Σ
<code>get_random_bytes_user</code>	—	—	—	—	—	4	4
<code>get_random_bytes</code>	81	—	—	4	2	13	100
<code>get_random_u8</code>	10	—	—	1	—	—	11
<code>get_random_u16</code>	7	—	2	—	1	1	11
<code>get_random_u32</code>	29	1	5	—	2	6	43
<code>__get_random_u32_below</code>	26	3	1	—	2	6	38
<code>get_random_u64</code>	—	5	—	—	1	4	10
Σ	153	9	8	5	8	34	217

G Generalizing the Attack

We perform data-flow analysis (DFA) on the Linux kernel to answer two questions:

Q1: What RNG state is “sensitive”—i.e., what state, when corrupted, influences random draws?

Q2: Where does the Linux kernel draw randomness?

For our analysis, we used the default, x86-64 kernel configuration shared by our case studies (§5) and evaluation (§7).

Sensitive RNG State. First, we consider **Q1**. Using SVF [149], we took a backward data-flow slice of the kernel’s RNG hierarchy starting at `chacha_block_generic`—responsible for computing a ChaCha20 block and writing the results to an output keystream—and walking back to the noise sources that contribute randomness to the input entropy pool. Given this scoped setting, we restrict our analysis to `drivers/char/random.c` and its dependencies (e.g., `lib/crypto/chacha.c`), allowing us to efficiently use SVF’s field-sensitive, inclusion-based (i.e., Andersen-style) points-to analysis [10]. Further, our analysis separates sliced data into two categories: *persistent* state, such as the L1 base DRNG’s key, and *transient* state, such as stack variables that are short-lived but, if corrupted, influence RNG output. After running the SVF pass, we manually verified and filtered the results, which are presented in Table 7. We omitted data that is outside our attack scope: e.g., data that only affects boot-time randomness and initial entropy collection and data that affects

how an individual noise source adds randomness to the input entropy pool. As the results in Table 7 show, “sensitive” data can be corrupted to perform data-only and control-flow hijacking attacks or affect availability. Further, there is a collection of transient state that can be (ab)used by attackers to affect extracted randomness or even disable SMAP [45]. Table 7 also records, for each entry, whether RNGuard isolates it (§6); the rows left unprotected are either neutralized by our hardening assumptions (§3) or fall outside RNGuard’s output-determining scope.

Random Draw Locations. To answer Q2, we enumerate *where* randomness is drawn using an LLVM pass over IR of the whole Linux kernel (linked together into a single bitcode file). The pass is a pure call-graph scan that iterates every instruction in every function looking for calls to one of the RNG API functions. The results for the default, x86-64 kernel configuration are shown in Table 6. For each kernel-specific API function, we provide a high-level classification of its call sites according to subsystem. Out of 217 call sites, networking is by far the largest consumer of randomness in the kernel (153 call sites), followed by the miscellaneous *other* category that groups process creation, UUID generation, RNG core functionality, and more (34 call sites). Further, the flexible `get_random_bytes`—which allows a caller to fill an arbitrarily sized buffer with randomness—is the most widely used API function (100 call sites), trailed closely by `get_random_u32` and `__get_random_u32_below` (81 call sites, collectively); the former extracts randomness directly from the L2 DRNG, while the latter extracts from the L3 batch pools. Notably, Table 6 does not illustrate the *frequency* of random draws, it only shows where they occur.

Table 7: Attacker-reachable kernel RNG state and RNGuard’s coverage of it.

State	Effect ^a	Prot. ^d	Notes
Persistent state			
<code>base_rng</code>	—	—	L1 base DRNG
<code>.key</code>	DO	✓	ChaCha20 key
<code>.generation</code>	DO	✓	counter directing L2, L3, and vDSO refresh
<code>crngs^b</code>	—	—	L2 per-CPU DRNGs
<code>.key</code>	DO	✓	ChaCha20 key most outputs are derived from
<code>.generation</code>	DO	✓	counter for syncing with L1 generation
<code>batched_entropy_uX^{b,c}</code>	—	—	L3 batch pools
<code>.entropy</code>	DO	✓	cached L2 DRNG output
<code>.generation</code>	DO	✓	counter for syncing with L1 generation
<code>.position</code>	DO	✓	current offset in cached randomness buffer
<code>_vdsso_rng_data</code>	—	—	vvar data for vDSO <code>getrandom()</code>
<code>.is_ready</code>	DO	✗ ^e	write true once to always bypass vDSO
<code>.generation</code>	DO	✗ ^e	counter for syncing with L1 generation
<code>input_pool</code>	—	—	input entropy pool
<code>.hash</code>	DO	✗ ^f	controllable if HW randomness is weak/absent
<code>next_reseed</code>	—	—	delayed work item used for periodic reseeding
<code>.work.data</code>	DO, A	✗ ^g	set pending bit to prevent further reseeding
<code>.work.func</code>	CFH, A	🔒	invoke <code>func</code> and prevent further reseeding
<code>pm_notifier</code>	—	—	power-management notifier
<code>.notifier_call</code>	CFH	🔒	invoke function pointer on system resumption
Transient state			
<code>crng_reseed</code>	—	—	extracts a fresh key into L1
<code>↪ key</code>	DO	✓	writes L1 key
<code>extract_entropy</code>	—	—	HKDF-like draw from <code>input_pool</code>
<code>↪ seed</code>	DO	✓	BLAKE2s digest reused as the PRF key
<code>↪ next_key</code>	DO	✓	next <code>input_pool</code> chaining key
<code>↪ block</code>	DO	✓	PRF salt (RDSEED counter)
<code>_get_random_bytes</code>	—	—	core byte generator
<code>↪ chacha_state</code>	DO	✓	ChaCha20 state the output is built from
<code>↪ tmp</code>	DO	✓	intermediate keystream block
<code>get_random_bytes_user</code>	—	—	backs /dev/urandom and <code>getrandom()</code>
<code>↪ chacha_state</code>	DO	✓ ^h	ChaCha20 state the output is built from
<code>↪ block</code>	DO	✓ ^h	keystream block copied to user
<code>crng_make_state</code>	—	—	returns a ChaCha state plus seed bytes
<code>↪ crng</code>	DO	✓	redirect to read another CPU’s DRNG
<code>↪ chacha_state</code>	DO	✓	state handed back to the caller
<code>crng_fast_key_erasure</code>	—	—	fast key-erasure ratchet
<code>↪ first_block</code>	DO	✓	controls erasure and first output block
<code>get_random_uX^e</code>	—	—	batched integer fast path
<code>↪ ret</code>	DO	✓	returned random integer
<code>↪ batch</code>	DO	✓	selects which L3 batch pool is used
<code>chacha_block_generic</code>	—	—	ChaCha core permutation
<code>↪ x</code>	DO	✓	working state matrix
<code>↪ i</code>	DO	✓	loop index; halts output write-back mid-loop
<code>__x64_sys_getrandom</code>	—	—	<code>getrandom()</code> syscall entry
<code>↪ iter</code>	DO	✗	iterator over the user output buffer
<code>*_irqsave callers</code>	—	—	every <code>spin_lock_irqsave</code> site
<code>↪ flags</code>	DO	✗	saved RFLAGS → clear AC to defeat SMAP

^a DO = data-only; CFH = control-flow hijack; A = availability

^b per-CPU data structures

^c X can be one of 8, 16, 32, or 64

^d ✓ = isolated by RNGuard; 🔒 = neutralized by the CFI hardening assumption (§3); ✗ = reachable, but not output-determining

^e no key material; user-readable by construction

^f entropy sources out of scope (§3)

^g deliberately excluded (§6.1)

^h eliminated by restructuring: output is produced per-chunk inside a protection window, so no live state persists across `copy_to_iter()`

ⁱ operates on delivered output, not RNG state

^j kernel-wide SMAP-bypass primitive, not RNG-specific